

An open educational platform

PRG32 — RISC-V Gaming (32-bit)

ESP32-C6 hardware • ESP32-C3 QEMU emulation

Playing with RISC-V

Assembly, C, and Retro Games on the PRG32 Platform

A First-Year Textbook for Computer Architecture
and Computer Programming

Built on the PRG32 educational runtime
Platform for RISC-V Gaming development (32-bit)

Academic supervision: Raffaele Montella, University of Naples “Parthenope”
Student contributors: Ivan Cafiero and Simone Boscaglia, University of Naples “Parthenope”

Every example in this book runs on both Espressif QEMU (desktop) and the physical ESP32-C6 board.

Companion repository: <https://github.com/riscv-prg32/PRG32>

Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform.

A first-year course companion for the Computer Architecture (assembly language) and Computer Programming (C language) curricula.

This textbook is built around the PRG32 open educational platform, an educational runtime for RISC-V assembly and C games that targets the ESP32-C6 RISC-V microcontroller for physical hardware and an ESP32-C3 Espressif QEMU target for desktop emulation. The platform, its source code, and its documentation are distributed under the MIT License by its authors.

The PRG32 software is the work of Raffaele Montella (academic supervisor and project lead), Ivan Cafiero (student contributor) and Simone Boscaglia (student contributor) at the University of Naples “Parthenope”. Readers who use PRG32 in coursework, reports, theses, or teaching material are asked to cite the project using the `CITATION.cff` metadata shipped in the repository ([Montella et al., 2026f](#)).

RISC-V is a registered trademark of RISC-V International. ESP32, ESP32-C3, ESP32-C6, and ESP-IDF are products and trademarks of Espressif Systems. All other trademarks are the property of their respective owners and are used here for identification only.

Edition compiled June 5, 2026. Typeset in L^AT_EX with Computer Modern.

Contents

Preface	vii
How to Read This Book	ix
I Foundations	1
1 Why a Game, Why RISC-V, Why Now	3
1.1 The problem this book solves	3
1.2 The platform in one paragraph	3
1.3 RISC-V as strategic European intellectual property	4
1.4 Why retro games, and what <i>Ready Player One</i> understood	5
1.5 Both targets, no compromise	6
1.6 How the rest of the book is arranged	6
2 Game Design Principles and the PRG32 Framework	9
2.1 What a game actually is	9
2.2 The three-function contract	10
2.3 The screen, the colours, and the coordinate system	10
2.4 Reading the player: the input bitmask	11
2.5 Drawing: the graphics calls you will use most	12
2.6 Sound: a beep when something happens	12
2.7 Cartridges: how a game reaches the machine	13
2.8 The two targets, concretely	13
2.9 Design principles for first games	14
2.10 A pedagogical map of the example games	14
3 Setting Up Your Development Environment	17
3.1 The one idea behind every setup	17
3.2 Environment installation	17
3.2.1 Windows	17
3.2.2 Linux	18
3.2.3 macOS	18
3.3 Physical board install	19
3.3.1 Windows	19
3.3.2 Linux	19
3.3.3 macOS	19
3.3.4 Build and flash	19

3.4	QEMU install	20
3.4.1	macOS	20
3.4.2	Linux	20
3.4.3	Install the Risc-V QEMU tool	20
3.5	Build and run	20
3.5.1	Windows	21
3.5.2	Linux and MacOS	21
3.6	Confirming it works	22
 II The Computer Architecture Track: RISC-V Assembly		23
4	First Contact: Registers, Memory, and the Call	25
4.1	The machine you are programming	25
4.2	The smallest program that runs	26
4.3	Cartridge building and uploading	27
4.3.1	ESP32-C6 hardware	27
4.3.2	QEMU	27
4.4	Simple instructions and program structure	28
4.4.1	Arithmetic and bit operations	28
4.4.2	Load and store word	28
4.4.3	Pseudo-instructions	29
4.5	Calling a C helper: the stack frame	29
4.6	Assembly sections: the anatomy of a program	31
4.7	Values in memory: the <code>.data</code> section	31
4.8	Putting it together: a console game that ticks	32
5	State and Input: Branches, Bitmasks, and Boundaries	35
5.1	Decisions: the conditional branch	35
5.2	Reading the controller	36
5.3	Moving an object with the joystick	36
5.4	Staying on the screen: the boundary clamp	39
5.5	Debugging like a systems programmer	39
6	Drawing the Game: Graphics, Collision, and the Frame	41
6.1	A blank frame	41
6.2	Drawing a rectangle	42
6.3	The ball: position, velocity, and the bounce	42
6.4	Did they touch? Rectangle collision	43
6.5	Assembling the complete game	44
6.6	Step-by-step: how this game was built	48
7	Capstone: Extending and Shipping an Assembly Game	49
7.1	Reading is a skill: Breakout as Pong with bricks	49
7.2	Extending your own game	50
7.3	Building a cartridge	50
7.4	When it goes wrong	50

III	The Computer Programming Track: The C Language	53
8	First Contact: Functions, Variables, and the Game Loop in C	55
8.1	Why C, and why the same platform	55
8.2	The three functions, in C	55
8.3	State: variables that persist between frames	56
8.4	Reading input: the bitmask in C	57
8.5	The ball: velocity and bounce, in C	58
8.6	Collision: one call, one decision	58
8.7	Drawing the frame	58
9	A Bigger World: Arrays, Structs, and Tile Maps	63
9.1	Arrays: many values under one name	63
9.2	Structs: grouping what belongs together	64
9.3	A tile map: the world as a grid	65
9.4	Behaviour from tile flags	65
9.5	Putting it together: an actor in a world	65
10	Capstone: Extending and Shipping a C Game	69
10.1	Embedding a C game in the firmware	69
10.2	Reusing helpers and reading richer examples	70
10.3	Arithmetic without floating point: fixed point	70
10.4	Building and shipping a C cartridge	71
IV	Bringing It Together	73
11	Assembly Meets C: One Game, Two Altitudes	75
11.1	The same logic, two languages	75
11.2	What C hides, and what it does not	76
11.3	A side-by-side map	76
11.4	Where to go next	77
V	Appendices	79
A	The PRG32 Runtime Interface	81
A.1	The calling convention	81
A.2	Lifecycle and timing	81
A.3	Input	82
A.4	Graphics	82
A.5	Sprites and collision	83
A.6	Playfields and tiles	83
A.7	Platform-game physics	83
A.8	Console and sound	84
A.9	Cartridges and scores	84
A.10	The cartridge tool	84
A.11	The HTTP runtime API	84
B	Measuring Performance Like a Scientist	87
B.1	Why measure, and what stays unchanged	87

B.2	What each frame records	87
B.3	The reproducible workflow	88
B.4	Getting results out	89
B.5	A complete profiling exercise	89
C	Full Working Examples	91
C.1	Pong in graphics assembly	91
C.2	Pong in ASCII assembly	95
C.3	Pong in C	97
C.4	Breakout in C	99
C.5	Where to find the rest	100
D	Using GitHub for Your Coursework	103
D.1	Why version control, concretely	103
D.2	Getting the platform	103
D.3	The everyday cycle	104
D.4	Branches: experiment freely	104
D.5	Recovering from mistakes	105
D.6	Submitting your work	105
D.7	Citing the platform	105
E	Building the Physical PRG32	107
E.1	The parts	107
E.2	Display and input wiring	107
E.3	Optional high-quality audio	108
E.4	Controllers over USB	109
E.5	From breadboard to board	109

Preface

There is an old and stubborn problem in the first year of every computer science and computer engineering degree. Two courses run in parallel, and they seem to belong to two different universes. In the computer architecture course, students meet registers, instructions, and the bare metal of a processor; the work is precise, but the reward is often nothing more than a number printed on a terminal. In the programming course, students meet variables, functions, and the comfortable abstractions of a high-level language; the work feels powerful, but the machine underneath has vanished from view. A student can pass both courses and still not feel, in any concrete way, that the `addi` instruction they traced by hand on Monday is the same machine that ran their C program on Thursday.

This book was written to dissolve that separation, and it does so with a deliberately old-fashioned trick: it makes a game move on a screen. When a white rectangle slides left because a student set a bit in a register, the distance between the instruction and the consequence collapses to nothing. The screen is honest in a way a grade is not. If the paddle leaves the viewport, the boundary check was wrong, and no amount of confident explanation will put it back. Retro game development turns the abstract machinery of a processor into something a beginner can see, hear, and argue about.

The platform that makes this possible is PRG32, an open educational runtime for RISC-V assembly and C games ([Montella et al., 2026f](#)). PRG32 is deliberately not a CPU emulator that hides the processor behind a comfortable layer. Student code runs *natively* as 32-bit RISC-V machine code, either on a real ESP32-C6 board on the desk or on Espressif's QEMU build on a laptop. The same three functions `init`, `update` and `draw` structure every example, whether it is written in hand-tuned assembly or in plain C. Because the two language tracks call exactly the same runtime, a teacher can show the same game logic at two altitudes: as a C function that reads almost like prose, and as a sequence of RISC-V instructions whose every register can be traced. That dual view is the pedagogical heart of this book.

Two practical commitments run through every chapter. First, every example is designed to work on *both* QEMU and the physical ESP32-C6 board, so a student without hardware is never locked out and a student with hardware is never limited to simulation. Second, the work is incremental and visible. The intended rhythm, borrowed directly from the PRG32 learning path, is small and concrete: change one thing, run it, observe the result, and write down what changed. Nothing in this book asks a beginner to type a hundred lines of assembly before seeing a single pixel.

A word on why the processor at the centre of this book matters beyond the classroom. RISC-V is an open instruction set architecture, free of the licensing restrictions that govern the proprietary architectures that have dominated computing for decades. For Europe in particular, that openness has become a matter of strategic technological sovereignty, and

the introduction that follows makes the case for treating RISC-V literacy as an investment, not a curiosity. Teaching a first-year student to read and write RISC-V is, in a small way, participation in that larger project.

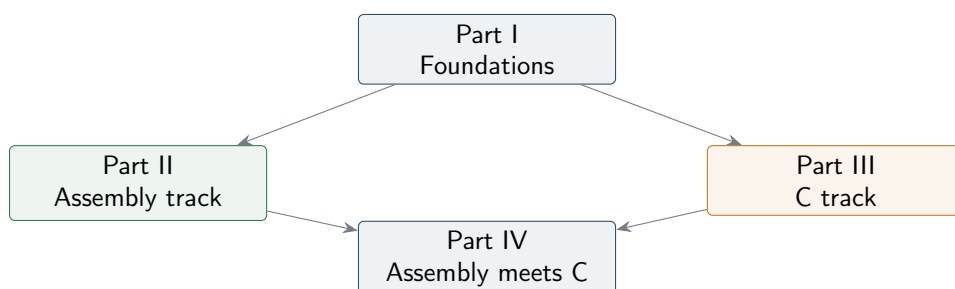
The book is organised so that the two courses it serves can use it independently or together. The computer architecture track and the computer programming track occupy separate parts and never depend on each other's internals, exactly as the PRG32 teaching guide recommends ([Montella et al., 2026h](#)). A short shared foundation at the start, and a short reunion at the end, are the only places the two paths touch. The appendices collect the reference material: the full runtime API, the performance-measurement workflow, complete worked examples, the use of GitHub, the construction of the physical board, and the configuration of the development environment on Windows, Linux, and macOS. This way, the chapters can stay focused on teaching.

I hope that by the last page a student will not merely have passed two courses, but will have built something that moves, makes noise, and runs on a chip they can hold in their hand, and will understand, instruction by instruction, why it works.

How to Read This Book

Two tracks, one runtime

This book contains two courses that share a single platform. Part II is the **Computer Architecture track**, written entirely in RISC-V assembly. Part III is the **Computer Programming track**, written entirely in C. The two parts are independent: neither requires you to have read the other, and they deliberately solve some of the same problems so that they can be compared. If you are following only one course, read the shared foundation in Part I, then your own track, and consult the appendices as needed. Part IV is a short closing chapter that brings the two languages into contact; it rewards readers who have seen both, but it is optional.



The recurring boxes

Throughout the book a small set of coloured boxes carries specific kinds of information. Learning to recognise them at a glance will speed up your reading.

Concept

A **Concept** box states an idea you should be able to explain in your own words before moving on. Concepts are the things an exam, or a colleague, will ask you about.

Try It Yourself

A **Try It Yourself** box is a small, concrete task. The book is built on the principle that you learn by changing one thing and observing the result, so these boxes appear often and are meant to be done, not merely read.

Checkpoint

A **Checkpoint** box lists what should be true if the preceding section worked. If a checkpoint fails, stop and fix it before continuing; later sections assume the earlier ones succeeded.

Watch Out

A **Watch Out** box flags a mistake that beginners reliably make. Reading these will save you hours of confusion at the workbench.

Historical Note

A **Historical Note** box steps back to place an idea in its context: why a convention exists, where a technique came from, or what an old machine did. These are enrichment; skipping them costs you nothing technical.

Both targets, every time

Every program in this book is designed to run on two targets without changing a single line of game code: Espressif's **QEMU** emulator on your laptop, and the physical **ESP32-C6** board. When a section gives build commands, it gives them for both. The QEMU target uses an ESP32-C3 emulator profile because that is the RISC-V graphics path Espressif maintains, while the hardware target is the ESP32-C6; the RISC-V calling convention and the PRG32 interface are identical across the two ([Montella et al., 2026e](#)). Wherever a feature exists only on real hardware: physical buttons, the buzzer, Wi-Fi cartridge upload, the text says so plainly.

What you need

To follow the book you need a computer running Windows, Linux, or macOS, and the free, open-source toolchain described in Appendix F. To follow the optional hardware exercises you also need an ESP32-C6 development board and a small collection of inexpensive parts listed in Appendix E. Nothing else in this book is proprietary or paid.

Part I

Foundations

CHAPTER 1

Why a Game, Why RISC-V, Why Now

1.1 The problem this book solves

A first-year student of computing is, in a sense, asked to believe two stories at once. The first story is told in the computer architecture course: a processor is a finite collection of registers and a small repertoire of instructions, and everything a computer does is, at bottom, a long sequence of those instructions moving numbers between registers and memory. The second story is told in the programming course: software is built from variables, functions, loops, and the clean abstractions of a high-level language, and the machine underneath is a detail best left to compilers. Both stories are true. The difficulty is that a beginner is rarely given a single object in which both stories are visibly the same object seen from two sides.

This book provides that object: a small retro video game running on a real RISC-V processor. In the assembly track you will write the game one instruction at a time and watch a rectangle move because you set a bit. In the C track you will write the same game as ordinary functions and watch the same rectangle move. Because both tracks compile to native RISC-V machine code and run on the same platform, the gap between the two stories closes. The compiler stops being a black box and becomes a translator whose output you can read.

Concept

The central pedagogical claim of this book is that a visible, audible, native program collapses the distance between an instruction and its consequence. A grade is an opinion; a paddle that leaves the screen is a fact. We will use facts.

1.2 The platform in one paragraph

The platform is PRG32, an open educational runtime for RISC-V assembly and C games ([Montella et al., 2026f](#)). It is important to understand what PRG32 is *not*: it is not a CPU instruction emulator that interprets your code on top of another machine. Your code runs natively, as 32-bit RISC-V machine code, either on a physical ESP32-C6 board or on Espressif's QEMU firmware target. Games are distributed as small *cartridges*, files with the extension `.prg32`, which the resident firmware loads and runs through three functions you will write yourself: an `init` called once, and an `update` and a `draw` called once per frame. That three-function shape is the entire contract between your game and the machine, and it is identical whether you write in assembly or in C. We will return to it in Chapter 2; for

now it is enough to know that the platform is deliberately thin, so that the processor it runs on is never far out of view.

1.3 RISC-V as strategic European intellectual property

It would be possible to teach this material on any number of processors. We have chosen RISC-V deliberately, and the reason reaches well beyond the classroom.

For most of the history of modern computing, the instruction set architecture, which is the fundamental contract that defines what a processor’s instructions mean, has been proprietary. The dominant architectures of the desktop, server, and mobile eras were owned by a small number of companies, and designing a competitive processor meant either licensing one of those architectures under restrictive terms or being shut out entirely. An instruction set is the deepest layer of a computing stack at which a nation or a region can exercise genuine technological independence, and for decades that layer was not open.

RISC-V changed the terms of the question. It is an open standard instruction set architecture, built on established reduced-instruction-set principles and governed as a shared specification rather than a private product ([RISC-V International, 2024](#)). Anyone may design, manufacture, and sell a RISC-V processor without paying a licensing fee for the architecture itself. That openness is not merely a convenience for hobbyists; it has become a matter of explicit industrial strategy, and nowhere more visibly than in Europe.

Historical Note

The name “RISC-V”, pronounced “risk-five”, marks the fifth generation of reduced-instruction-set computing research at the University of California, Berkeley, where the architecture began as an academic project. That it grew from a teaching and research instruction set into a strategic industrial standard is a useful reminder that the line between a classroom exercise and a national technology base is thinner than it looks.

The European Union has come to treat RISC-V as a strategic lever for technological sovereignty and for greater competition in a global semiconductor market worth on the order of seven hundred billion United States dollars ([Wohlrab, 2026](#)). Europe currently accounts for roughly a tenth of that market, and the 2023 European Chips Act sets the explicit goal of doubling that share to a fifth by the end of the decade ([Wohlrab, 2026](#)). Achieving such a goal on borrowed, proprietary architectures would leave the continent’s most strategic hardware dependent on decisions made elsewhere; building it on an open architecture does not.

This is not aspiration on paper alone. Through the EuroHPC Joint Undertaking, Europe has launched the DARE project: “Digital Autonomy with RISC-V in Europe”, a multi-year, multi-partner effort, backed by funding on the order of two hundred and forty million euros, to design European processors and accelerators for high-performance computing and artificial intelligence on a RISC-V foundation ([EuroHPC Joint Undertaking, 2025](#); [Digital Autonomy with RISC-V in Europe \(DARE\) Consortium, 2025](#)). DARE builds on a lineage of earlier European programmes, including the European Processor Initiative, and is paralleled by industrial efforts such as the TRISTAN project under the Chips Joint Undertaking, whose partners have assembled Europe’s first comprehensive collection of verified, industry-ready RISC-V intellectual property ([Wohlrab, 2026](#)). The recurring word in all of these

announcements is *sovereignty*: the capacity to build strategic computing technology without asking permission.

Concept

RISC-V is open intellectual property: a freely usable instruction set rather than a licensed product. Europe has made it a centrepiece of its strategy for technological sovereignty, funding processor and high-performance-computing programmes built on it. Learning to read and write RISC-V is therefore not only a foundational computing skill but a small act of participation in that strategy.

For a first-year student the lesson is concrete and encouraging. The instructions you will trace by hand in Chapter 4: `addi`, `lw`, `sw` and `beqz` are not a teaching dialect invented for a course. They are the same instructions that European industry and academia are using to build the processors that the continent intends to depend on. The skill transfers directly, and it transfers to a technology whose importance is still increasing.

1.4 Why retro games, and what *Ready Player One* understood

If RISC-V answers the question “which processor,” retro game development answers the question “which kind of program.” The choice is, again, deliberate.

Retro games occupy a pedagogical sweet spot. They are small enough that a beginner can hold an entire program in their head, yet rich enough to exercise every concept a first course needs: state stored in memory, input read from the world, arithmetic that updates that state, branches that make decisions, and a loop that ties it all together at a fixed frame rate. A game written for a 320-by-200 retro viewport, with a handful of moving rectangles, is simultaneously a complete and satisfying artefact and a transparent specimen for study. Crucially, a game gives *immediate, honest feedback*. A sorting routine that is subtly wrong may still print plausible numbers; a game that is subtly wrong puts the ball through the paddle, and the error announces itself.

There is also a cultural argument, and it is here that Ernest Cline’s novel *Ready Player One* is more than a passing reference (Cline, 2011). The novel imagines a future in which fluency with the games, machines, and idioms of an earlier computing era becomes the most valuable literacy of all: the key, in the story quite literally, to a vast inheritance. Stripped of its science-fiction framing, the book captures something true about computing as a discipline: the foundational layers do not become obsolete so much as they become invisible, and the people who can still see them retain a kind of power. The student who understands how a sprite is drawn pixel by pixel, how a collision is detected with a few comparisons, and how a frame is timed, understands the substrate on which all the comfortable modern abstractions are built. Retro game development is, in this sense, a way of preserving and transmitting exactly the literacy that *Ready Player One* treats as priceless.

Historical Note

The aesthetic of this book: the 320-by-200 viewport, the chunky rectangles, the single-channel beep, the cartridge that you “insert” by uploading a `.prg32` file, is a faithful homage to the home computers and consoles of the late twentieth century. The constraints are not nostalgia for its own sake. A tight memory budget and a simple display force a beginner to understand what is actually happening, in a way that a modern engine with gigabytes of memory and a forgiving runtime never would.

1.5 Both targets, no compromise

A practical commitment shapes every example in this book: it must run on both the desktop emulator and the physical board. The emulator, Espressif’s QEMU build, lets you compile, run, and debug graphics on a laptop with no hardware at all; it is ideal for checking layout, movement, timing, and for stepping through code with a debugger. The physical ESP32-C6 board is where the program meets the real world: real buttons, a real display, a real buzzer, and wireless cartridge upload ([Montella et al., 2026e](#)). The QEMU target is configured as an ESP32-C3 emulator profile because that is the RISC-V graphics path Espressif maintains, while the hardware is the ESP32-C6; the RISC-V calling convention and the PRG32 interface do not change between them, so the same source builds for both.

Both targets, every example. Throughout the book this badge marks code that has been designed to run unchanged on QEMU and on the ESP32-C6. Where a feature is hardware-only (physical buttons, the buzzer, Wi-Fi upload) or emulator-only (host-side debugging conveniences), the text says so explicitly.

1.6 How the rest of the book is arranged

Part I, which this chapter opens, sets the foundation. Chapter 2 introduces game design principles and the PRG32 framework in detail; it is the one chapter that every reader, on either track, should read before going further.

Part II is the computer architecture track, in RISC-V assembly. It begins with your first program, builds through state and input, develops a complete graphical game, and ends with a capstone you assemble yourself.

Part III is the computer programming track, in C. It is structured to parallel Part II without depending on it: the same three-function shape, the same runtime, the same two targets, expressed in a high-level language.

Part IV is a short reunion. Having written games in both languages, you will look at the two side by side and see, concretely, what a compiler does.

The appendices are the reference shelf: the complete runtime interface, the performance-measurement workflow, full annotated example programs, a guide to using GitHub for your coursework, instructions for building the physical board, and step-by-step environment setup for Windows, Linux, and macOS. The chapters send you to the appendices rather than interrupting the narrative with reference tables.

Checkpoint

Before continuing, you should be able to explain, in a sentence or two each: why this book uses a game as its teaching object; what it means to say RISC-V is an open instruction set and why Europe treats it as strategic; and what it means that every example runs on both QEMU and the ESP32-C6.

CHAPTER 2

Game Design Principles and the prg32 Framework

This is the one chapter that belongs to both tracks. Whether you are here for assembly or for C, you need a shared mental model of two things: how a small game is structured as a program, and how the PRG32 framework turns that structure into something you can see and hear. Read it once now; you will return to it often.

2.1 What a game actually is

Strip away the graphics and a game is a very old idea: a loop that never ends until you tell it to. Each turn around the loop is called a *frame*. In a single frame the machine does three things, always in the same order. It *reads* the state of the world: which buttons are pressed, how much time has passed. It *updates* an internal model of the game, moves the ball, checks for collisions, adjusts the score. And it *draws* the new state to the screen. Then it does it again, fast enough that the eye perceives motion rather than a slideshow.

Concept

A game is a loop over frames. In each frame the program reads input, updates its state, and draws the result. “State” is simply the set of variables that describe the game right now: positions, velocities, scores, lives. Everything interesting a game does is a rule for turning this frame’s state into next frame’s state.

It is worth dwelling on the discipline this imposes, because it is the discipline PRG32 enforces. Update and draw are kept separate on purpose. Update is allowed to change state but should not draw; draw is allowed to read state and put pixels on the screen but should not change the game’s logic. A beginner who mixes the two ends up with games where, for instance, a collision is detected only when an object happens to be drawn, and bugs of that kind are maddening to find. Keeping the phases separate is not bureaucracy; it is the single most effective habit for writing games that behave predictably.

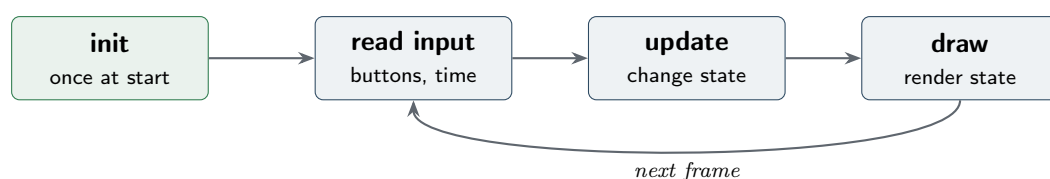


Figure 2.1. The frame loop. PRG32 calls your `init` once, then your `update` and `draw` once per frame, forever.

2.2 The three-function contract

PRG32 expresses the frame loop as a contract of exactly three functions that you provide and the framework calls (Montella et al., 2026h). Every example in this book, in either language, exports the same three entry points, distinguished only by a prefix that names the game:

```
<prefix>_init      -- runs once when the game (cartridge) is
                    selected
<prefix>_update    -- runs once per frame; changes game state
<prefix>_draw      -- runs once per frame; renders the current
                    state
```

Listing 2.1. The universal shape of a PRG32 game. The prefix changes; the structure never does.

For a Pong game written in assembly the prefix is `pong_graphics`, so the symbols are `pong_graphics_init`, `pong_graphics_update`, and `pong_graphics_draw`. For the C version the prefix is `pong_c`. The framework does not care which language produced the symbols; it only needs the three of them.

Concept

The entire interface between your game and PRG32 is three functions: `init`, `update`, and `draw`. Learn this shape once and every example in the book, assembly or C, emulator or hardware, becomes a variation on it.

2.3 The screen, the colours, and the coordinate system

PRG32 presents a display that is 320 pixels wide and 240 pixels tall, but games draw into a centred *viewport* of 320 by 200, leaving thin bands at the top and bottom for status information (Montella et al., 2026f). The coordinate origin (0,0) is the top-left corner; x increases to the right and y increases *downward*. This downward- y convention surprises newcomers who remember graphs from mathematics, where y goes up, and it is one of the most common sources of early confusion.

Watch Out

On the screen, y grows *downward*. A larger y means lower on the display. If your object moves up when you expected it to move down, you have very likely confused the sign of a y change. This is normal; check it first.

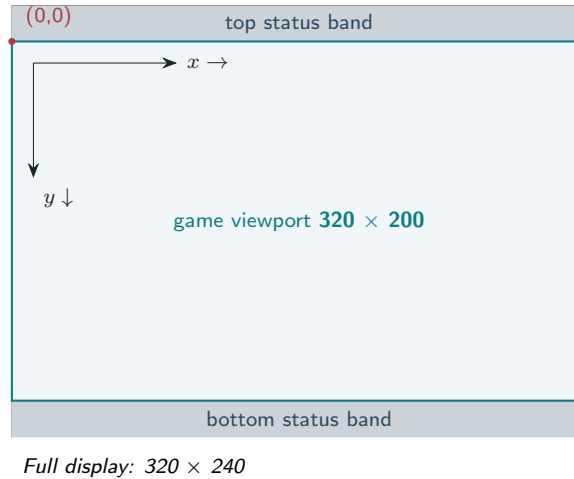


Figure 2.2. The PRG32 display. Framework screens use the full 320×240 ; your game draws into the centred 320×200 viewport.

Colours are expressed in the **RGB565** format: a 16-bit number that packs five bits of red, six of green, and five of blue. You rarely need to compute these by hand, because the framework defines names for the common ones: `PRG32_COLOR_BLACK` is `0x0000`, `PRG32_COLOR_WHITE` is `0xffff`, `PRG32_COLOR_YELLOW` is `0xffe0`, and so on (Montella et al., 2026f). When you see the literal `65535` in an assembly example, it is simply white written in decimal.

Historical Note

RGB565 was the natural pixel format of countless devices in the era this book celebrates, precisely because it fits a pixel in two bytes while still giving the eye enough colour. The extra green bit reflects the fact that human vision is most sensitive to green. The format is a small, elegant compromise between memory and fidelity: exactly the kind of constraint that makes retro programming instructive.

2.4 Reading the player: the input bitmask

PRG32 reports all the buttons at once as a single integer, a *bitmask*, in which each button is one bit (Montella et al., 2026f). Reading input is a single call, `prg32_input_read`, that returns this integer. To find out whether a particular button is pressed, you test its bit with a bitwise AND.

Constant	Button	Bit value
<code>PRG32_BTN_LEFT</code>	left	<code>0x01</code>
<code>PRG32_BTN_RIGHT</code>	right	<code>0x02</code>
<code>PRG32_BTN_UP</code>	up	<code>0x04</code>
<code>PRG32_BTN_DOWN</code>	down	<code>0x08</code>
<code>PRG32_BTN_A</code>	A	<code>0x10</code>
<code>PRG32_BTN_B</code>	B	<code>0x20</code>
<code>PRG32_BTN_SELECT</code>	select	<code>0x40</code>

Table 2.1. Player-one button bits. Player two uses the same layout shifted into higher bits (`PRG32_P2_*`).

The reason a bitmask is used, rather than seven separate variables, is that it mirrors how the hardware actually presents the buttons: as bits in a register. A student who understands the input bitmask understands, in miniature, how a processor reads any memory-mapped peripheral. We will use this fact in both tracks.

Watch Out

Test a button with bitwise AND (&), not equality (==). The mask may hold several pressed buttons at once, so `input & PRG32_BTN_LEFT` asks “is the left bit set, regardless of the others?”, which is what you want. Writing `input == PRG32_BTN_LEFT` would only be true when *exactly* left and nothing else is pressed.

2.5 Drawing: the graphics calls you will use most

The framework offers a small drawing interface, and a beginner needs only a handful of its functions. Each of the calls below works identically in assembly (passing arguments in registers) and in C (passing arguments normally), and each works identically on QEMU and on the ESP32-C6 (Montella et al., 2026e).

Call	What it does
<code>prg32_gfx_clear(color)</code>	fill the whole viewport with one colour
<code>prg32_gfx_pixel(x,y,color)</code>	set a single pixel
<code>prg32_gfx_rect(x,y,w,h,color)</code>	draw a filled rectangle
<code>prg32_gfx_text8(x,y,s,fg,bg)</code>	draw a string of text
<code>prg32_gfx_present()</code>	show the finished frame on the display
<code>prg32_sprite_hitbox(...)</code>	test whether two rectangles overlap

Table 2.2. The core drawing and collision calls. The full graphics interface is catalogued in Appendix A.

The rectangle call, `prg32_gfx_rect`, is the workhorse of early examples: a paddle is a wide rectangle, a ball is a small square, a wall is a tall thin rectangle. The collision call, `prg32_sprite_hitbox`, takes the position and size of two rectangles and returns a non-zero value when they overlap, which is exactly the test a beginner needs for “did the ball hit the paddle?”

2.6 Sound: a beep when something happens

Audio in PRG32 starts as simply as it can: `prg32_audio_beep(hz, ms)` plays a tone of a given frequency for a given number of milliseconds (Montella et al., 2026a). On the physical board this drives a small buzzer; on QEMU the call is accepted but silent, because the emulator does not model the buzzer. The framework’s own advice, which this book repeats, is to play a beep only on an *event*: a collision, a score, a button press. Don’t play it on every frame, because a tone restarted sixty times a second is an unpleasant buzz rather than a sound effect.

Watch Out

Do not call `prg32_audio_beep` every frame. Trigger sound on events only. A beep in `update` that fires whenever the ball is in flight will play continuously; a beep that fires only in the frame where a collision is newly detected will sound like a game.

2.7 Cartridges: how a game reaches the machine

A finished PRG32 game is packaged as a *cartridge*: a file ending in `.prg32` that contains your compiled RISC-V code together with a small header (Montella et al., 2026b). The resident firmware, flashed onto the board once, at the start of the course, knows how to load a cartridge and call its three functions. This separation is what lets you iterate quickly: you flash the firmware a single time, then upload cartridge after cartridge without reflashing, either over the board’s own Wi-Fi network or, on the emulator, by staging the cartridge into the QEMU flash image.

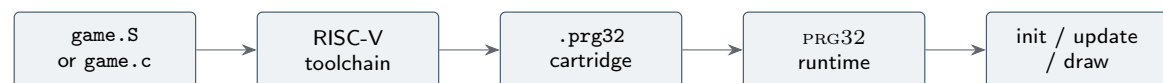


Figure 2.3. From source to running game. The same pipeline serves assembly and C, hardware and emulator (Montella et al., 2026f).

The cartridge header begins with the magic bytes `PRG2` and records, among other things, the offsets of the `init`, `update`, and `draw` functions, so the firmware knows where to jump (Montella et al., 2026b). You will not write this header by hand; a tool called `prg32_game.py` builds it for you, and Appendix A documents the tool’s command line. For now the important idea is conceptual: a cartridge is your three functions, compiled to native code, wrapped in just enough metadata for the firmware to run them.

2.8 The two targets, concretely

Every game in this book is built for two targets: the physical ESP32-C6 board and the QEMU emulator. The source code is the same for both; only the build directory and profile differ (Montella et al., 2026e).

The physical board uses `build-esp32c6` and the ESP32-C6 configuration. QEMU uses `build-qemu` and the ESP32-C3 emulator target, because the Espressif QEMU graphics path is mature for that chip. The two targets are different hardware, but from your game’s point of view the framework interface stays the same.

For the detailed workflow and the exact build commands, see the environment and build instructions in Chapter 3.

Watch Out

Keep the two build directories strictly separate. Flashing a `build-qemu` image to a real board can fail because the emulator build targets a different display path than the hardware. If the board shows nothing, the first thing to check is that you built in `build-esp32c6`, not `build-qemu` (Montella et al., 2026e).

Historical Note

The emulator uses ESP32-C3 while the board uses ESP32-C6, but both are RISC-V microcontrollers running the same 32-bit instruction set. The framework keeps the calling convention and game interface identical, so the difference is largely an implementation detail.

2.9 Design principles for first games

The PRG32 learning path distils years of classroom experience into a few rules, and they are worth stating plainly because beginners who follow them succeed and beginners who ignore them struggle (Montella et al., 2026f,i).

The first rule is *start with returns that do nothing*. Write `init`, `update`, and `draw` so that each simply returns immediately, build it, and confirm the framework runs without crashing. You now have a skeleton that compiles and runs; everything after is addition.

The second rule is *add one thing at a time*. One variable for position. Then draw one object at that position. Then read input and change the position. Then add one boundary check. Then one collision. Each step is small enough to verify by eye before the next begins.

The third rule is *sound comes last*. Get the core loop working and visible before adding audio, because a beep that fires at the wrong time is a symptom, not a feature, and you want the visual logic solid first.

Concept

The intended rhythm of work is small and visible: change one thing, run it, observe the result, and write down what changed (Montella et al., 2026f). This is not merely advice for beginners; it is how experienced systems programmers actually work when the machine is unforgiving.

2.10 A pedagogical map of the example games

The platform ships with a family of complete example games, each provided in three forms: an ASCII assembly version, a graphics assembly version, and a C version (Montella et al., 2026f). They are arranged, roughly, from simplest to richest, and this book draws its worked examples from them in that order.

Example	What it teaches first
<code>pong</code>	state, input, boundaries, one collision
<code>breakout</code>	arrays of objects, removing objects on hit
<code>space_invaders</code>	many moving objects, simple enemy waves
<code>pacman</code>	grid movement, a tile map
<code>asteroids</code>	wrap-around motion, rotation
<code>tetris</code>	falling pieces, a fixed grid, line clears
<code>platformer</code>	tile flags, gravity, camera follow
<code>raycaster</code>	fixed-point arithmetic, a pseudo-3D corridor
<code>wing_commander</code>	two layered playfields, a cockpit

Table 2.3. The example games, ordered by the concepts they introduce. This book builds Pong from scratch in both tracks and uses the others as references ([Montella et al., 2026f,h](#)).

We will build Pong together, line by line, in both the assembly track (Chapters 4–7) and the C track (Chapters 8–10), because it is the smallest game that still contains every essential idea. The richer examples then become readable: once you have written Pong, the Breakout source is Pong with an array of bricks, and the platformer source is Pong with gravity and a tile map.

Checkpoint

Before starting a track, make sure you can answer these without looking back: What are the three functions every PRG32 game provides? In which direction does y grow on the screen? How do you test whether one button is pressed? What is a cartridge? Why must the QEMU and board builds use separate directories?

CHAPTER 3

Setting Up Your Development Environment

This chapter helps you get a built, runnable PRG32 on Windows, Linux, or macOS. The instructions follow the platform's own setup guides ([Montella et al., 2026f](#)). The common foundation on every operating system is the same: install Espressif's ESP-IDF, the development framework for these RISC-V microcontrollers, then build the firmware for your chosen target. Pick your operating system's section, then read the shared sections on QEMU and the build commands.

3.1 The one idea behind every setup

Whatever your operating system, the toolchain is ESP-IDF version 5.3 or newer, configured for the `esp32c3` and `esp32c6` targets: the first for the QEMU emulator, the second for the physical board ([Montella et al., 2026f,e](#)). Once ESP-IDF is installed and "sourced" (made active in your shell), the command `idf.py` builds, flashes, and runs everything. Almost every setup problem reduces to ESP-IDF not being active in the current shell.

Watch Out

ESP-IDF must be activated in *each new terminal* before `idf.py` works. If you see `idf.py: command not found` or a missing `riscv32-esp-elf-gcc`, ESP-IDF is simply not sourced in that shell. This is the single most common setup error, and the fix is always the same: re-run the export step ([Montella et al., 2026f](#)).

3.2 Environment installation

Install ESP-IDF and any platform prerequisites first. After ESP-IDF is active, `idf.py` becomes available in the shell and the rest of this workflow can run.

3.2.1 Windows

The platform recommends a classroom path built on Visual Studio Code with the Espressif ESP-IDF extension, which installs the toolchain for you, plus the PlatformIO and Microsoft C/C++ and Python extensions ([Montella et al., 2026f](#)). There is also a standalone path using the official ESP-IDF Tools Installer, after which you work from the "ESP-IDF PowerShell" shortcut it creates.

```
cd $HOME\Documents
git clone https://github.com/riscv-prg32/PRG32.git
cd PRG32
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
set -target esp32c6
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
build
```

Listing 3.1. Windows standalone path, run from the ESP-IDF PowerShell shortcut.

Watch Out

On Windows, use the ESP-IDF PowerShell or ESP-IDF Command Prompt, not a plain terminal where `idf.py` has not been exported. If flashing fails, check Device Manager for the board's serial port and pass it explicitly with `-p COMx`; if the monitor cannot open the port, close every other program using it (Arduino Serial Monitor, other ESP-IDF monitors, and so on) ([Montella et al., 2026f](#)).

3.2.2 Linux

On Debian or Ubuntu, install the build prerequisites with the system package manager, then ESP-IDF as above ([Montella et al., 2026f](#)). Other distributions use their own package names for the equivalent libraries.

```
sudo apt update
sudo apt install -y git wget flex bison gperf python3 python3-
    venv \
    python3-pip cmake ninja-build ccache libffi-dev libssl-dev \
    dfu-util libusb-1.0-0
cd $HOME
git clone -b v5.3 --recursive https://github.com/espressif/esp-
    idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
```

Listing 3.2. Linux (Debian/Ubuntu) prerequisites, then ESP-IDF.

3.2.3 macOS

The platform's macOS path, tested on Apple Silicon, installs the prerequisites with Homebrew and then ESP-IDF ([Montella et al., 2026f](#)).

```
brew install git cmake ninja dfu-util ccache libusb python
cd $HOME
git clone -b v5.3 --recursive https://github.com/espressif/esp-
    idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
```

Listing 3.3. macOS setup with Homebrew and ESP-IDF.

A convenient habit is an alias so a single word activates ESP-IDF in any shell ([Montella et al., 2026f](#)).

```
alias get_idf=". $HOME/esp-idf/export.sh"
```

Listing 3.4. A quality-of-life alias for activating ESP-IDF.

3.3 Physical board install

The physical ESP32-C6 board uses the ESP-IDF toolchain for the `esp32c6` target. Make sure your board is connected, ESP-IDF is active, and you have serial-port access configured for your OS.

3.3.1 Windows

Use the ESP-IDF PowerShell or ESP-IDF Command Prompt created by the installer. Once ESP-IDF is active, the same firmware build and flash commands work.

3.3.2 Linux

To flash a real board, your user must be allowed to use the serial port. Add yourself to the `dialout` group, then log out and back in ([Montella et al., 2026f](#)).

```
sudo usermod -aG dialout "$USER" # then log out and back in
```

Listing 3.5. Granting serial-port access on Linux.

Historical Note

ESP32-C6 boards usually appear as `/dev/ttyACM0`, while USB-bridge boards may appear as `/dev/ttyUSB0` ([Montella et al., 2026f](#)). If flashing fails although the board is visible, the cause is almost always the `dialout` membership not yet taking effect. Log out and back in after adding yourself to the group.

3.3.3 macOS

On macOS, once ESP-IDF is active the physical board flash workflow is the same as on the other platforms. Use the native serial device exposed by the board and flash it from an active ESP-IDF shell.

3.3.4 Build and flash

With ESP-IDF active, build and flash the firmware for the physical board using the following commands. On the **physical board**, the resident firmware starts its own Wi-Fi access point

named PRG32. You connect your computer to that network and upload the cartridge over HTTP; the firmware stores it and runs it, no reflashing required.

```
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
set-target esp32c6
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
flash monitor
```

Listing 3.6. Building and flashing for the physical ESP32-C6 board.

3.4 QEMU install

Installing QEMU lets you run the PRG32 platform on the desktop using the `esp32c3` target.

Linux and MacOS hosts need a few native libraries for the QEMU display window ([Montella et al., 2026e](#)).

3.4.1 macOS

```
brew install libgcrypt glib pixman sdl2 libslirp
```

Listing 3.7. QEMU window libraries for MacOS.

3.4.2 Linux

```
sudo apt-get install -y libgcrypt20 libglib2.0-0 libpixman-1-0
libSDL2-2.0-0 libslirp0
```

Listing 3.8. QEMU window libraries for Linux.

3.4.3 Install the Risc-V QEMU tool

Install Espressif’s RISC-V QEMU after ESP-IDF, then re-activate ESP-IDF so the new tool is on the path ([Montella et al., 2026e](#)).

```
python $IDF_PATH/tools/idf_tools.py install qemu-riscv32
. $IDF_PATH/export.sh
```

Listing 3.9. Installing the RISC-V QEMU tool and re-activating ESP-IDF.

3.5 Build and run

You will build every game for two targets. The commands differ only in a build directory and a configuration profile; the source does not change at all ([Montella et al., 2026e](#)). Make sure that ESP-IDF is active by sourcing `export.sh` in the `esp-idf` directory.

For the **physical ESP32-C6 board**, the build uses a directory named `build-esp32c6` and the board configuration profile.

```
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
set-target esp32c6
idf.py -B build-esp32c6 -D SDKCONFIG_DEFAULTS=sdkconfig.defaults
flash monitor
```

Listing 3.10. `building_and_flashing_esp32c6.sh`

Building and flashing for the physical board (abbreviated; see Chapter 3).

For the **QEMU emulator**, the build uses `build-qemu` and the QEMU profile, and targets the ESP32-C3 emulator. On **QEMU**, there is no Wi-Fi to upload over, so you build the cartridge against the QEMU firmware ELF and stage it directly into the emulator’s flash image. If the flash image does not yet exist, run QEMU once so ESP-IDF creates it, quit, then stage ([Montella et al., 2026e](#)).

3.5.1 Windows

```
idf.py -B build-qemu -D SDKCONFIG_DEFAULTS=sdkconfig.defaults.
qemu set-target esp32c3
idf.py -B build-qemu -D SDKCONFIG_DEFAULTS=sdkconfig.defaults.
qemu qemu --graphics monitor
```

Listing 3.11. `building_and_flashing_qemu_windows.ps1`

Building and running for the QEMU emulator on Windows.

3.5.2 Linux and MacOS

```
./scripts/qemu/build_qemu.sh
```

Listing 3.12. `building_and_flashing_qemu_shell.sh`

Building and running for Linux or MacOS.

Watch Out

Keep the two build directories strictly separate. Flashing a `build-qemu` image to a real board produces a black screen, because the emulator build uses a virtual display panel that does not exist on real hardware. If your board shows nothing, the first thing to check is that you built in `build-esp32c6`, not `build-qemu` ([Montella et al., 2026e](#)).

Historical Note

Why does the emulator target an ESP32-C3 while the board is an ESP32-C6? Both are RISC-V microcontrollers from the same family, and both run the same 32-bit RISC-V instructions. Espressif maintains a mature QEMU graphics path for the ESP32-C3, so PRG32 uses it for desktop emulation, while the physical kits are built around the newer ESP32-C6 ([Montella et al., 2026e](#)). From your game’s point of view the difference is invisible: the calling convention and the framework interface are identical.

3.6 Confirming it works

A healthy board boot logs the firmware entering `app_main`, the configured display pins, and finally "PRG32 runtime initialized," then shows the splash ([Montella et al., 2026i](#)). The platform also ships a smoke test that exercises the whole workflow: environment, firmware build, cartridge build, and QEMU staging, and prints `=== SMOKE TEST PASSED ===` when all is well ([Montella et al., 2026f](#)).

```
./scripts/smoke_test.sh
```

Listing 3.13. Running the end-to-end smoke test.

Checkpoint

Your environment is ready when, on your operating system, `idf.py` runs in an ESP-IDF shell, the QEMU build opens a virtual screen reaching "PRG32 runtime initialized," and, if you have a board, the firmware flashes and shows its splash. With that in place, every chapter and every example in this book is buildable on your machine.

Part II

The Computer Architecture Track: RISC-V Assembly

CHAPTER 4

First Contact: Registers, Memory, and the Call

This chapter begins the computer architecture track. By its end you will have a running RISC-V assembly program on both QEMU and, if you have one, the physical board: a program that clears a console, prints text, and makes a sound. More importantly, you will understand the three ideas on which everything else rests: what a register is, how a value lives in memory, and what happens when you call a function. We build these ideas on the smallest possible game, the ASCII version of Pong, exactly as the platform’s own tutorial does ([Montella et al., 2026j](#)).

4.1 The machine you are programming

A RISC-V processor of the kind in your board is, from the programmer’s point of view, astonishingly simple. It has thirty-two registers, each holding a 32-bit value, and it executes instructions that, with few exceptions, do one of three things: an arithmetic or logical operation between registers, a transfer of a value between a register and memory, or a change in which instruction runs next. That is very nearly the whole machine. The richness of computing comes not from a large instruction set but from combining a small one in long sequences.

Concept

A register is a tiny, extremely fast storage location inside the processor that holds one 32-bit value. The processor can only compute on values that are in registers. To work on a value stored in memory, you must first *load* it into a register; to keep a result, you must *store* it back to memory.

The thirty-two registers have conventional roles, fixed not by the hardware but by an agreement called the *calling convention* or *ABI*. Following the convention is what lets your assembly code and the framework’s C code call each other. Table 4.1 shows the registers you will actually use; the full convention is in Appendix A.

Register	Name	Role in this book
a0–a7	arguments	values passed into and out of calls; a0 also holds the return value
t0–t6	temporaries	scratch values; a call may destroy them
s0–s11	saved	values a call promises to preserve
ra	return address	where a <code>ret</code> jumps back to
sp	stack pointer	top of the stack; kept 16-byte aligned
zero	zero	always reads as 0

Table 4.1. The RISC-V registers you will use, and their agreed roles (Montella et al., 2026a).

Concept

The **calling convention** is an agreement about which registers carry arguments (a0–a7), which carry results (a0), and which survive a function call (s0–s11, but not t0–t6). PRG32 follows the standard RISC-V convention, which is exactly why an assembly game can call a C helper such as `prg32_gfx_clear` (Montella et al., 2026i).

4.2 The smallest program that runs

Recall the three-function contract from Chapter 2. The smallest legal PRG32 game provides `init`, `update`, and `draw` that each do nothing but return. In RISC-V assembly, “return” is the instruction `ret`, and “do nothing else” is simply not writing anything else. We name our game `hello` and give every symbol that prefix.

```

1  .option norelax
2  .section .text
3  .global hello_init
4  .global hello_update
5  .global hello_draw
6
7  hello_init:
8      ret                # return to the framework
9
10 hello_update:
11     ret
12
13 hello_draw:
14     ret

```

Listing 4.1. `hello_world_asm.S`

The smallest PRG32 assembly game: three functions that return immediately. Build it, confirm it runs, then add to it.

A few details deserve explanation. The directive `.option norelax` tells the assembler not to apply an optimisation that can interfere with the way PRG32 cartridges are linked; every example begins with it (Montella et al., 2026l). The `.section .text` directive says that what follows is executable code, as opposed to data. The `.global` directives make the three symbols visible to the framework, which must find them by name. Without `.global`, your functions would exist but the loader could not see them.

Checkpoint

Build this skeleton for QEMU and confirm it runs without crashing. You will see no game yet: that is correct. The achievement is that the framework found your three functions, called them, and survived. Chapter 3 has the exact build commands for your operating system; Appendix C shows this skeleton wired into the firmware.

4.3 Cartridge building and uploading

The repository contains a quick-start guide in `README.md` and documentation in the `docs/` folder. For convenience, scripts and tools are provided to build cartridges, stage them for QEMU, or upload them to a board over WiFi.

4.3.1 ESP32-C6 hardware

To upload a cartridge to a real board, build it first and then use the included Python tool. Note that the board IP is `192.168.4.1` by default.

```
python3 tools/prg32_game.py build <PATH_IN.S|c> \
--out <PATH_OUT.prg32> \
--firmware-elf build-esp32c6/PRG32.elf \
--entry-prefix <CARTRIDGE_PREFIX> \
--name <CARTRIDGE_NAME> \

python3 tools/prg32_game.py upload <PATH_OUT.prg32> --url http
://192.168.4.1
```

Listing 4.2. Examples: build and upload (Wi-Fi and QEMU)

4.3.2 QEMU

To stage a cartridge for QEMU, build it, inject it into the image, and then launch the emulator.

```
./scripts/cartridge/build_cartridge.sh <PATH_IN.S|c> <PATH_OUT.
prg32> <GAME_PREFIX>
./scripts/qemu/qemu_inject_cartridge.sh <PATH_OUT.prg32>
./scripts/qemu/launch_qemu.sh
```

Listing 4.3. Build and upload a cartridge to QEMU

These commands work on both macOS and Linux. For additional details, see `README.md` and the files in `docs/` (for example `docs/qemu.md`).

Both targets. The cartridge format is identical; what changes is the firmware ELF you build against (`build-esp32c6` versus `build-qemu`) and the delivery (`upload` over Wi-Fi versus `upload-qemu` into the flash image). Your game source is byte-for-byte the same in both cases (Montella et al., 2026e).

Watch Out

Build the cartridge against the matching firmware ELF. A cartridge built against `build-esp32c6/PRG32.elf` resolves the framework symbols at the addresses the board firmware uses; staging that same cartridge into a QEMU image built at different addresses will not work. Build for the target you intend to run on.

4.4 Simple instructions and program structure

A working program is built from a few very small operations. These operations are provided by the RISC-V Instruction Set Architecture ([RISC-V International, 2024](#)). We will now introduce the foundational operations you need for writing your first programs.

4.4.1 Arithmetic and bit operations

The first group is the arithmetic and bitwise instructions. In a real program these are used to update counters, combine values, compute flags, and prepare arguments for later work. They all read values from registers and write the result back into a register. The result is stored in the first register.

Instruction	What it does	Example
<code>add</code>	Add two registers	<code>add t0, t1, t2</code>
<code>sub</code>	Subtract one register from another	<code>sub t0, t1, t2</code>
<code>xor</code>	Bitwise exclusive-or of two registers	<code>xor t0, t1, t2</code>
<code>or</code>	Bitwise OR of two registers	<code>or t0, t1, t2</code>
<code>and</code>	Bitwise AND of two registers	<code>and t0, t1, t2</code>

The RISC-V ISA also provides immediate forms of many of these operations. These instructions take one register operand and one small constant, which is useful for adding a fixed amount or applying a fixed mask in one step.

Instruction	What it does	Example
<code>addi</code>	Add a register and a constant	<code>addi t0, t1, 4</code>
<code>xori</code>	Exclusive-or a register with a constant	<code>xori t0, t1, 1</code>
<code>ori</code>	OR a register with a constant	<code>ori t0, t1, 0xff</code>
<code>andi</code>	AND a register with a constant	<code>andi t0, t1, 7</code>

4.4.2 Load and store word

A second group of instructions moves whole 32-bit values between memory and registers. In a real program these are the only instructions that touch the state stored in the program data; they let the program remember values from one frame to the next.

Instruction	What it does	Example
<code>lw</code>	Load a 32-bit word from memory into a register	<code>lw t0, 0(t1)</code>
<code>sw</code>	Store a 32-bit word from a register into memory	<code>sw t0, 0(t1)</code>

Each load or store combines a register holding a memory address with a small constant offset. That makes it easy to access fields inside a data structure or variables laid out in memory.

You can use a zero offset when you don't need it.

4.4.3 Pseudo-instructions

The final group is the pseudo-instructions. They are not separate hardware operations; instead, they are convenient shortcuts that the assembler expands into one or more real instructions. In everyday code they make assembly easier to read and write.

Pseudo	What it does	Example
la	Load the address of a symbol into a register	<code>la a0, title</code>
li	Load a constant value into a register	<code>li t0, 42</code>
mv	Copy one register to another	<code>mv t0, t1</code>
not	Bitwise complement of a register	<code>not t0, t1</code>

In practice, **la** is used whenever a program needs the address of a string or a data variable. **li** is used to load a number directly into a register without first storing it in memory. **mv** is simply a copy, and **not** computes the one's complement of a register value.

With these instruction families, the RISC-V ISA provides the basic operations a program needs: arithmetic and bitwise logic, memory load/store, and the pseudo-instructions that make assembly manageable. The next section shows how this instruction set is used when your assembly calls into the framework.

4.5 Calling a C helper: the stack frame

A game that only returns is not interesting for long. The first useful thing we can do is print text, using the framework's console. The console is written in C, so calling it means crossing the boundary between your assembly and the framework's C, and that boundary has one rule a beginner must internalise.

When you execute **call**, the processor records, in the register **ra**, the address it should return to. But the C function you are calling will itself use **ra** for its own calls, overwriting yours. If you do nothing to protect your **ra**, then when your function tries to **ret**, it will jump to the wrong place and your program will misbehave or crash. The solution is to save **ra** on the *stack* before the call and restore it afterwards.

Concept

The **stack** is a region of memory that grows downward and is used to save values across function calls. Before calling a C helper, an assembly function lowers the stack pointer **sp** to make room, stores **ra** there, makes its call, then loads **ra** back and raises **sp**. This save/restore pair is the single most important habit in RISC-V assembly.

Here is the pattern, which you will repeat in nearly every function:

```

1 .option norelax
2 .section .text
3
4 .global print_init
5 .global print_update
6 .global print_draw

```

```

7
8 print_init:
9     addi sp, sp, -16    # make 16-byte frame (keep sp 16-aligned)
10    sw   ra, 12(sp)     # save return address into the frame
11    call prg32_console_clear # clear text console (C helper)
12    la   a0, title      # put address of string in a0 (1st argument)
13    call prg32_console_write # print it
14    lw   ra, 12(sp)     # restore the return address
15    addi sp, sp, 16     # tear down the frame
16    ret                # now ret goes to the right place
17
18 print_update:
19     ret
20
21 print_draw:
22     ret
23
24 .section .rodata
25 title:
26     .asciz "HELLO PRG32\n" # a zero-terminated string

```

Listing 4.4. `print_asm.S`

The stack-frame idiom. Save `ra`, do work that may call C, restore `ra`, return.

Read this slowly, because it contains five distinct ideas. The `addi sp, sp, -16` subtracts sixteen from the stack pointer, reserving sixteen bytes; we use a multiple of sixteen because the convention requires `sp` to stay 16-byte aligned across calls ([Montella et al., 2026a](#)). The `sw ra, 12(sp)` stores the word in `ra` at an offset of twelve bytes above the new stack top. The `la a0, title` loads the *address* of our string into `a0`, because the convention puts the first argument in `a0` and the console function expects a pointer. The two `call` instructions invoke C helpers. Finally the load and `addi` undo the save and the reservation, in the reverse order, before `ret`.

Watch Out

Forgetting `sw ra` before a `call`, or `lw ra` after it, is the classic first bug. The symptom is a function that “returns to the wrong place”: the program jumps somewhere unexpected, often crashing. If your assembly behaves strangely around a call, check the `ra` save/restore pair first ([Montella et al., 2026j](#)).

Historical Note

Why does the stack grow *downward*, toward smaller addresses? It is a convention almost as old as the stored-program computer, and it lets the stack and the program’s data grow toward each other from opposite ends of memory, making the most of a fixed address space. Subtracting from `sp` to allocate, and adding to free, follows directly from that choice.

4.6 Assembly sections: the anatomy of a program

When writing assembly, you must explicitly tell the assembler how to categorize different parts of your program using directives. The `.section` keyword is the switch that selects where the following code or data belongs: `.data` for mutable state, `.rodata` for constants, and `.text` for executable instructions. The `.data` section is for mutable state—variables like our counter that the program must modify during execution. Conversely, `.rodata` stands for “read-only data” and is reserved for constants, such as our game title string. On physical hardware, placing constants in `.rodata` allows the toolchain to keep them safely locked in flash memory so they cannot be accidentally overwritten by a bug.

The very first directive in a PRG example is often `.option norelax`. That tells the assembler not to apply its relaxation pass, so it does not rewrite certain branch or symbol addresses later. For PRG32 cartridges, the exact layout of code and data matters, and `.option norelax` helps keep that layout predictable.

While section directives dictate *where* data lives in memory, the `.global` directive dictates *who* can see it. By default, every label you define in an assembly file is strictly private to that file. By marking a label as `.global`, you export it into the project’s global symbol table. This step is what bridges your assembly code to the rest of the ecosystem, explicitly giving the game framework permission to locate, link against, and jump into your functions from the outside.

4.7 Values in memory: the `.data` section

A game must remember things between frames: where the ball is, what the score is. In assembly, persistent values live in the `.data` section, each given a label and an initial value. This is the assembly equivalent of a global variable. For ASCII Pong, the platform’s example declares its state exactly this way ([Montella et al., 2026f](#)):

```
1 .section .data
2 hello_x:      .word 20      # object x position, in text columns
3 hello_score:  .word 0      # a simple counter
```

Listing 4.5. `game_state_data.S`

Game state in `.data`. Each label names a memory location holding one 32-bit word.

To use such a value you do two things, and the distinction between them is the crux of understanding memory. First you load the *address* of the label into a register with `la`. Then you load the *value* at that address with `lw`, or store a new value with `sw`.

```
1 la t0, hello_x      # t0 = ADDRESS of hello_x
2 lw t1, 0(t0)        # t1 = VALUE stored at that address
3 addi t1, t1, 1      # change the value (in the register)
4 sw t1, 0(t0)        # write the new value back to memory
```

Listing 4.6. `load_modify_store.S`

The load–modify–store cycle: read a value from memory, change it in a register, write it back.

Concept

A label such as `hello_x` names a *location* in memory. `la t0, hello_x` puts that location's address in `t0`; `lw t1, 0(t0)` reads the *contents* of that location into `t1`. The processor computes only on contents, in registers, then writes results back. This load–modify–store cycle is how every variable in every program ultimately works.

Watch Out

Keep the two clearly apart in your mind: `t0` holds an *address*, `t1` holds a *value*. Confusing them, using an address where a value belongs, or vice versa, produces wrong numbers and is one of the most common beginner errors. A useful habit, used throughout the platform's examples, is to comment each register with whether it holds an address or a value.

4.8 Putting it together: a console game that ticks

We now have every piece needed for a minimal but genuine program: an `init` that prints a title, an `update` that advances a counter each frame, and a `draw` that prints the score to screen. We also included a short beep sound that works with the `prg32_audio_beep` call. (Montella et al., 2026j).

```

1  .option norelax
2  .section .data
3      counter: .word 0
4
5  .section .rodata
6      title: .asciz "HELLO PRG32\n"
7
8  .section .text
9  .global counter_init
10 .global counter_update
11 .global counter_draw
12
13 counter_init:
14     addi sp, sp, -16
15     sw   ra, 12(sp)           # save return address
16
17     li   a0, 440              # 440 Hz: a short beep on start
18     li   a1, 60               # 60 ms duration
19     call prg32_audio_beep     # silent on QEMU, audible on the
20                               board
21
22     lw   ra, 12(sp)
23     addi sp, sp, 16
24     ret
25
26 counter_update:
27     addi sp, sp, -16
28     sw   ra, 12(sp)           # save return address
29     la   t0, counter

```

```

30     lw    t1, 0(t0)
31     addi  t1, t1, 1           # count one tick per frame
32     sw    t1, 0(t0)
33
34     lw    ra, 12(sp)         # load return address
35     addi  sp, sp, 16
36     ret
37
38 counter_draw:
39     addi  sp, sp, -16
40     sw    ra, 12(sp)         # save return address
41
42     call  prg32_console_clear
43
44     la    a0, title
45     call  prg32_console_write
46
47     la    t0, counter
48     lw    a0, 0(t0)
49     call  prg32_console_hex32 # print numeric value
50
51     lw    ra, 12(sp)
52     addi  sp, sp, 16         # load return address
53     ret

```

Listing 4.7. counter.S

A counter example that displays an incrementing score.

This short program demonstrates the core design pattern of every game on this platform. Notice how the work is split across distinct phases: `counter_update` handles the logic by modifying our variable in memory, while `counter_draw` focuses purely on presentation. This separation of concerns is fundamental; it ensures that your game mechanics remain entirely independent of how or when the screen is drawn.

The program also highlights how data types behave at the machine level through our choice of console helpers. While `prg32_console_write` requires the memory address of a null-terminated ASCII string, `prg32_console_hex32` takes a raw 32-bit numeric value directly in `a0`. It reads the content of the register itself and formats that raw integer into human-readable text on the fly. Confusing these two—passing a raw number where an address is expected—is a classic recipe for a crash.

Finally, look at the entry and exit points of all three functions. Every single one opens with an identical prologue to lower `sp` and save `ra`, and closes with a matching epilogue to restore them. Because every function here uses a `call` instruction to hand off work to the framework, they would all overwrite their own return path without this defensive housekeeping.

Both targets. On QEMU you will see the title on the serial monitor; the beep is accepted but silent because the emulator does not model the buzzer. On the ESP32-C6 you will additionally hear the beep through the buzzer. The *source is identical*; only the build directory and configuration profile differ, as shown in Chapter 2.

Try It Yourself

Change the beep frequency from 440 to 880 and rebuild. On the board, the tone rises by an octave. Then change the increment in `hello_update` from 1 to 2 and predict, before running, what the counter does. Verify your prediction. This “change one thing, predict, verify” loop is the method of the entire book ([Montella et al., 2026f](#)).

Checkpoint

You should now be able to build and run an assembly program on at least QEMU; explain why `ra` must be saved before a `call`; explain the difference between `la` and `lw`; and describe what the `.data`, `.text`, and `.rodata` sections are for. If any of these is shaky, re-read the relevant section before Chapter 5; everything ahead builds on them.

CHAPTER 5

State and Input: Branches, Bitmasks, and Boundaries

In Chapter 4 a counter advanced on its own. A game, though, responds to a player. This chapter teaches the assembly you need to read the controller and to make decisions: the input bitmask, the bitwise `andi` test, and the conditional branch. By the end you will move an object with the joystick and keep it on the screen: the core interactive loop of every game in this book. We follow the structure of the platform’s ASCII Pong update routine, which was written to be read by first-year students (Montella et al., 2026j,f).

5.1 Decisions: the conditional branch

So far our instructions ran in a straight line. To respond to the player, a program must choose: if a button is pressed, move; otherwise, do not. In RISC-V this choice is a *conditional branch*, an instruction that jumps to a label only when a condition holds.

Concept

A **conditional branch** compares registers and jumps to a label only if the comparison is true; otherwise execution simply continues to the next instruction. The branch is how a processor makes every decision, from “is this button pressed?” to “has the ball hit the wall?”

The branches you will use most are few. `beqz t, label` jumps if `t` is zero; `bltz t, label` jumps if `t` is negative; `ble a, b, label` jumps if $a \leq b$; and the unconditional `j label` always jumps. A common idiom, which surprises beginners, is to branch on the *opposite* of what you want: to do something “if the button is pressed,” you test the button bit and `beqz`, branch away, when it is *not* pressed, letting the movement code run only when it is. This reads backwards at first but becomes natural quickly.

Historical Note

Local labels written as a bare number followed by a colon, like `1:`, and referenced as `1f` (the next `1` “forward”) or `1b` (“backward”), are an assembler convenience for short jumps that need no memorable name. The platform’s examples use them heavily for the small skips inside an update routine (Montella et al., 2026f).

5.2 Reading the controller

Recall from Chapter 2 that `PRG32` returns all buttons at once as a bitmask. In assembly you obtain it by calling `prg32_input_read`; the result arrives, by convention, in `a0`. The very first thing to do is copy it somewhere safe, because the next `call` you make would overwrite `a0`.

```
1  call prg32_input_read  # button bitmask returned in a0
2  mv   t0, a0           # keep our own copy in t0 before anything else
```

Listing 5.1. `get_input.S`

Read the input bitmask and keep it. The copy to a temporary is essential, because later calls reuse `a0`.

Watch Out

Copy the input mask out of `a0` immediately. `a0` is the argument and return register; the moment you call another function, its old contents are gone. The platform’s examples copy the mask into a temporary on the very next line for exactly this reason (Montella et al., 2026j).

To test a single button you isolate its bit with `andi`, the “and-immediate” instruction, using the bit value from Table 2.1. The result is non-zero exactly when that button is pressed.

```
1  andi t3, t0, 1        # PRG32_BTN_LEFT is bit 0 (value 1)
2  beqz t3, 1f           # if the LEFT bit is clear, skip the add
3  addi t1, t1, -1       # LEFT is pressed: move the object left
4  1: ...
```

Listing 5.2. `button_test.S`

Testing one button. `andi` keeps only the chosen bit; `beqz` skips the body when that bit is clear.

Concept

`andi t3, t0, 1` computes the bitwise AND of the input mask with the constant 1, leaving in `t3` the value of bit 0 alone. If `LEFT` is pressed, `t3` is non-zero; if not, it is zero. Testing a button is therefore “mask the bit, then branch on whether the result is zero.”

5.3 Moving an object with the joystick

We now assemble a complete `update` that moves an object left and right with the joystick. The state is a single word in `.data`, exactly as in Chapter 4. The routine reads input, tests `LEFT` and `RIGHT`, and updates the position. For now, we limit the draw to displaying the position on screen. In the future, the position will make a graphics object move

(Montella et al., 2026f).

```
1  .option norelax
```

```

2
3 .section .data
4 game_x: .word 5      # starting value of 5
5 text_update: .word 0
6
7 .section .rodata
8 program_title: .asciz "INPUT_READ\n"
9
10 .section .text
11 .global input_read_init
12 .global input_read_update
13 .global input_read_draw
14
15 input_read_init:
16     addi sp, sp, -16
17     sw    ra, 12(sp)
18
19     # we draw the title and score at the start
20
21     call prg32_console_clear
22
23     la    a0, program_title
24     call prg32_console_write # draw the title
25
26     la    t0, game_x
27     lw    a0, 0(t0)
28     # draw the starting x value
29     call prg32_console_hex32
30
31     lw    ra, 12(sp)
32     addi sp, sp, 16
33     ret
34
35 input_read_update:
36     addi sp, sp, -16
37     sw    ra, 12(sp)
38
39     call prg32_input_read # the result is stored in a0
40     mv    t0, a0 # t0 now stores the input result
41
42
43     la    t1, game_x # t1 = game_x address
44     lw    t2, 0(t1) # t2 = game_x value
45
46     andi t3, t0, 1 # test LEFT (bit 0)
47     beqz t3, 1f
48     addi t2, t2, -1 # move left
49
50     # when an update occurs, we want to take note
51     # that the screen should update
52     li    t4, 1
53     la    t5, text_update
54     sw    t4, 0(t5)
55
56 1: andi t3, t0, 2 # test RIGHT (bit 1)
57     beqz t3, 2f

```

```

58     addi t2, t2, 1    # move right
59
60     li    t4, 1
61     la    t5, text_update
62     sw    t4, 0(t5)
63
64 2:   sw    t2, 0(t1)      # store the new x
65
66     lw    ra, 12(sp)
67     addi sp, sp, 16
68     ret
69
70 input_read_draw:
71     addi sp, sp, -16
72     sw    ra, 12(sp)
73
74     la    t0, text_update # we check if an update has occurred
75     lw    t1, 0(t0)
76     beqz t1, 2f # when nothing happens, we skip the draw
77
78     call prg32_console_clear
79
80     la a0, program_title
81     call prg32_console_write
82
83     la t0, game_x
84     lw a0, 0(t0)
85     call prg32_console_hex32 # redraw the current position
86
87     li    t1, 0          # clear the update flag once drawn
88     la    t0, text_update
89     sw    t1, 0(t0)
90
91 2:   lw    ra, 12(sp)
92     addi sp, sp, 16
93     ret

```

Listing 5.3. A full update: read input, move on LEFT/RIGHT, store the result. Compare with the platform’s `pong_ascii_update`.

Trace it once by hand. Suppose `game_x` holds 20 and the player holds RIGHT. The input read returns a mask with bit 1 set. The LEFT test (`andi ... 1`) yields zero, so `beqz` skips the left move. The RIGHT test (`andi ... 2`) yields non-zero, so the right move runs and `t2` becomes 21. The store writes 21 back. Next frame it becomes 22, and so on. The object slides right as long as RIGHT is held. This kind of by-hand trace is the most valuable debugging skill you will build in this course.

The code also introduces a tiny performance trick. Instead of redrawing the screen on every update, it stores a small flag in `text_update` whenever LEFT or RIGHT actually changes `game_x`. The draw routine checks this flag and skips the clear/write sequence when nothing moved. In the previous chapter, the example redrew continuously, but a real game should avoid doing work when the state does not change.

There is also something to notice about the numbers on screen. If you move left past zero, the value underflows and the display shows `0xffffffff` instead of a normal count. If you

move right past 9, the console printer keeps counting in hexadecimal, so 10 appears as **a**, 11 as **b**, and so on. That is because we are displaying the 32 bit score word is displayed as an hexadecimal value.

5.4 Staying on the screen: the boundary clamp

Left unchecked, our object will slide right off the edge of the screen and vanish. The fix is a *clamp*: after computing the new position, force it back into the legal range. For the 40-column ASCII console the legal range is 0 to 39; for the 320-pixel graphics viewport it will be 0 to a value that accounts for the object's width. The logic is identical either way.

```

1      bltz t2, 3f          # if x < 0, clamp to the left edge
2      li   t3, 39         # maximum column
3      ble  t2, t3, 4f      # if x <= 39, it is already in range
4      mv   t2, t3         # otherwise clamp to the right edge
5      j    4f
6  3:  li   t2, 0           # left clamp
7  4:  sw   t2, 0(t1)       # store the bounded x

```

Listing 5.4. . Negative values snap to 0; values past the edge snap to the maximum. | `x_clamping.S`
Clamping x to the range [0, 39]. Negative values snap to 0; values past the edge snap to the maximum.

Concept

A **boundary clamp** keeps a value inside a legal range: if it went below the minimum, set it to the minimum; if above the maximum, set it to the maximum; otherwise leave it alone. Almost every off-screen bug in a beginner's game is a missing or wrong clamp. When an object disappears, suspect the clamp first.

Watch Out

The maximum is not the screen width; it is the screen width minus the object's own size. A 64-pixel-wide paddle on a 320-pixel screen may go no further right than $320 - 64 = 256$, or its right edge would leave the viewport. The platform's graphics Pong clamps the paddle to exactly 256 for this reason (Montella et al., 2026f).

Try It Yourself

Add the clamp above to your `update`. Write one sentence explaining which comparison protects the right edge. This break-and-fix exercise is taken directly from the platform's labs (Montella et al., 2026l).

5.5 Debugging like a systems programmer

When an assembly game misbehaves, the platform's guidance is explicit and worth adopting as a discipline: do not guess, trace state (Montella et al., 2026i). Ask, in order, which register held the last correct value, which instruction changed it, whether a `call` overwrote a temporary you expected to keep, and whether a branch skipped the code that updates

memory. Reduce the program to the smallest version that still shows the bug. Nine times in ten the answer is one of three things: a missing `ra` save, an address-versus-value confusion, or a branch that goes the wrong way.

Checkpoint

You should now be able to read the input bitmask into a register; test an individual button with `andi` and `beqz`; move a value in `.data` in response to the joystick; and clamp that value to a legal range. With these, you have written the interactive core of a game. Chapter 6 adds the one missing ingredient: drawing it on the screen.

CHAPTER 6

Drawing the Game: Graphics, Collision, and the Frame

You can now move an object with the joystick and keep it on the screen. This chapter gives that object a body. We move from the text console to the 320-by-200 graphics viewport, draw rectangles for a paddle and a ball, detect when they touch, and tie the three functions together into a complete, playable graphical Pong game. (Montella et al., 2026e,l).

6.1 A blank frame

We start with a `draw` that clears the screen to black and does nothing else: confirm it runs.

```
1  .option norelax
2
3  .section .text
4  .global blank_init
5  .global blank_update
6  .global blank_draw
7
8  blank_init:
9      ret
10
11 blank_update:
12     ret
13
14 blank_draw:
15     addi sp, sp, -16
16     sw    ra, 12(sp)
17
18     li    a0, 0                # RGB565 black
19     call  prg32_gfx_clear      # fill the 320x200 viewport
20
21     lw    ra, 12(sp)
22     addi  sp, sp, 16
23     ret
```

Listing 6.1. `hello_world_graphics.S`

A `draw` that clears the frame. Build this first and confirm a black screen before drawing anything on it.

6.2 Drawing a rectangle

The mental model from ASCII based programs is unchanged: `state` lives in `.data`, `update` changes it, `draw` renders it. What changes is the drawing call. Instead of `prg32_console_write`, we use `prg32_gfx_clear` to erase the frame and `prg32_gfx_rect` to draw a filled rectangle.

To draw a rectangle, we use the call `prg32_gfx_rect(x, y, w, h, color)`. In assembly the five arguments go, in order, into `a0` through `a4`, following the calling convention. The colour in `a4` is an RGB565 value; you may write it as a named constant in C, but in assembly it is usually a plain number: 65535 for white, 65504 for yellow, 0 for black (Montella et al., 2026f).

Watch Out

The five arguments of `prg32_gfx_rect` occupy `a0=x`, `a1=y`, `a2=width`, `a3=height`, `a4=colour`. Getting an argument into the wrong register draws the wrong rectangle: a vivid, immediate way to see what the calling convention actually means.

In the next full example we will draw a white paddle near the bottom of the viewport, at a position read from `.data`. The other dimensions are constant: we will draw a 64-by-8 white bar at `y = 188`. Wide enough to be easy to see in the emulator and perfect for recreating the old classic Pong. (Montella et al., 2026f). Here is what the paddle draw section will look like:

```

1  la    t0, paddle_x           # address of the paddle position
2  lw    a0, 0(t0)              # a0 = x (a value read from memory)
3  li    a1, 188                # a1 = y (near the bottom)
4  li    a2, 64                 # a2 = width
5  li    a3, 8                  # a3 = height
6  li    a4, 65535              # a4 = white (RGB565 0xffff)
7  call  prg32_gfx_rect         # draw the paddle

```

Listing 6.2. paddle_draw.S

Drawing the paddle from a stored position. The five arguments fill `a0–a4` in order.

6.3 The ball: position, velocity, and the bounce

Now we need a ball. The ball needs four words of state: its `x` and `y` position and its `x` and `y` velocity (which the examples call `dx` and `dy`). Each frame, the velocity is added to the position; when the ball reaches an edge, the velocity is negated so it travels back the other way. (Montella et al., 2026f).

```

1
2  # --- BALL X MOVEMENT ---
3      la    t1, ball_x
4      lw    t2, 0(t1)          # t2 = ball x
5      la    t3, ball_dx
6      lw    t4, 0(t3)          # t4 = ball dx
7      add   t2, t2, t4          # x = x + dx
8

```

```

9      # Check Left Edge (x < 0)
10     bgez t2, check_right_edge
11     li   t2, 0           # clamp to left edge
12     neg  t4, t4          # reverse direction
13     j    save_ball_x
14
15     check_right_edge:
16     li   t5, 312         # 320 - ball width (8)
17     ble  t2, t5, save_ball_x
18     mv   t2, t5          # clamp to right edge
19     neg  t4, t4          # reverse direction
20
21     save_ball_x:
22     sw   t2, 0(t1)       # store new x
23     sw   t4, 0(t3)       # store dx
24
25
26
27     # --- BALL Y MOVEMENT ---
28     la   t1, ball_y
29     lw   t2, 0(t1)       # t2 = ball y
30     la   t3, ball_dy
31     lw   t4, 0(t3)       # t4 = ball dy
32     add  t2, t2, t4      # y = y + dy
33
34     # Check Top Edge (y < 0)
35     bgez t2, save_ball_y
36     li   t2, 0           # clamp to top edge
37     neg  t4, t4          # reverse direction (bounce down)
38
39     save_ball_y:
40     sw   t2, 0(t1)       # store new y
41     sw   t4, 0(t3)       # store new dy

```

Listing 6.3. ball_motion.S

Autonomous ball motion on the x and y axis with edge bounce.

Concept

Velocity is just a number added to position each frame. A *bounce* is negating that number when the object reaches a boundary: `neg t4, t4` turns `+2` into `-2`, so the next frames move the ball back the way it came. With this one idea: add velocity and negate at the edges, you can make anything bounce.

6.4 Did they touch? Rectangle collision

A ball that passes straight through the paddle is no fun. We need to know when the ball's rectangle overlaps the paddle's rectangle. The framework provides exactly this test: `prg32_sprite_hitbox(ax, ay, aw, ah, bx, by, bw, bh)` takes the position and size of two rectangles and returns non-zero when they overlap (Montella et al., 2026l). In assembly the eight arguments fill `a0` through `a7`.

```

1  la    t0, ball_x
2  lw    a0, 0(t0)      # a0 = ball x
3  la    t0, ball_y
4  lw    a1, 0(t0)      # a1 = ball y
5  li    a2, 8          # a2 = ball width
6  li    a3, 8          # a3 = ball height
7
8  la    t0, paddle_x
9  lw    a4, 0(t0)      # a4 = paddle x
10 li    a5, 188         # a5 = paddle y
11 li    a6, 64         # a6 = paddle width
12 li    a7, 8          # a7 = paddle height
13
14
15 call  prg32_sprite_hitbox # a0 = non-zero if overlapping
16 beqz  a0, no_overlap    # no overlap: skip the bounce
17
18 # overlap found!
19 # play a sound
20 li    a0, 440          # 440 Hz: a short beep on start
21 li    a1, 60           # 60 ms duration
22 call  prg32_audio_beep # silent on QEMU, audible on the board
23
24 # reverse direction
25 la    t0, ball_dy
26 lw    t1, 0(t0)
27 blez  t1, no_overlap   # only bounce if ball is moving down (
                          # t1 > 0)
28 neg   t1, t1           # Reverse dy natively
29 sw    t1, 0(t0)

```

Listing 6.4. collision_detection.S

Testing ball-versus-paddle overlap. A non-zero result in a0 means they touched this frame.

Two rectangles overlap when they intersect on *both* axes at once: the *x* ranges overlap *and* the *y* ranges overlap. `prg32_sprite_hitbox` performs this two-axis test for you and returns the answer as a single value, so collision becomes one call followed by one branch.

Concept

Notice that the bounce reuses the clamp from Chapter 5, with one addition: where the clamp merely stopped the object at the edge, the bounce also negates the velocity. The platform's examples keep the clamp so the ball never visibly leaves the screen even for a single frame, then negate so it returns. Good game feel often comes from such small combinations of simple rules.

6.5 Assembling the complete game

We now have every part: a paddle moved by input and clamped, a ball that moves and bounces, and a collision test. Putting them in the three functions gives a complete graphical game whose structure exactly matches the platform's `pong_graphics` example ([Montella](#)

et al., 2026f). The `update` moves the paddle from input, advances and bounces the ball, and applies collision; the `draw` clears the frame and renders the paddle and ball.

```

1  .section .data
2      paddle_x: .word 140
3      ball_x: .word 80
4      ball_y: .word 80
5      ball_dx: .word 2      # positive dx means moving right
6      ball_dy: .word 2      # positive dy means moving down
7
8  .section .text
9  .global pong_skeleton_init
10 .global pong_skeleton_update
11 .global pong_skeleton_draw
12
13 pong_skeleton_init:
14     ret
15
16 pong_skeleton_update:
17     addi sp, sp, -16
18     sw    ra, 12(sp)      # save return address
19
20     # --- PADDLE INPUT AND X MOVEMENT ---
21     call prg32_input_read # read controller bits into a0
22     mv    t0, a0          # save input mask in t0
23
24     la    t1, paddle_x
25     lw    t2, 0(t1)       # t2 = paddle_x
26
27     andi  t3, t0, 1       # test LEFT (bit 0)
28     beqz  t3, 1f
29     addi  t2, t2, -3      # move paddle left
30 1:     andi  t3, t0, 2     # test RIGHT (bit 1)
31     beqz  t3, 2f
32     addi  t2, t2, 3       # move paddle right
33
34 2:     # clamp paddle_x to the visible range [0, 256]
35         # 256 = 320 - 64, the paddle_width
36
37     # --- LEFT CLAMP ---
38     bgez  t2, clamp_right # t2 >= 0, then check right side
39     li    t2, 0          # else, clamp to 0
40     j     store_paddle_x
41
42     # --- RIGHT CLAMP ---
43 clamp_right:
44     li    t3, 256
45     ble   t2, t3, store_paddle_x # t2 <= 256, the position is
46         saved.
47     li    t2, 256        # else, clamp it to 256.
48
49 store_paddle_x:
50     sw    t2, 0(t1)
51
52 # --- BALL X MOVEMENT ---

```

```

52     la    t1, ball_x
53     lw    t2, 0(t1)        # t2 = ball x
54     la    t3, ball_dx
55     lw    t4, 0(t3)        # t4 = ball dx
56     add   t2, t2, t4        # x = x + dx
57
58     # Check Left Edge (x < 0)
59     bgez  t2, check_right_edge
60     li    t2, 0            # clamp to left edge
61     neg   t4, t4           # reverse direction
62     j     save_ball_x
63
64 check_right_edge:
65     li    t5, 312          # 320 - ball width (8)
66     ble   t2, t5, save_ball_x
67     mv    t2, t5           # clamp to right edge
68     neg   t4, t4           # reverse direction
69
70 save_ball_x:
71     sw    t2, 0(t1)        # store new x
72     sw    t4, 0(t3)        # store dx
73
74 # --- BALL Y MOVEMENT ---
75     la    t1, ball_y
76     lw    t2, 0(t1)        # t2 = ball y
77     la    t3, ball_dy
78     lw    t4, 0(t3)        # t4 = ball dy
79     add   t2, t2, t4        # y = y + dy
80
81     # Check Top Edge (y < 0)
82     bgez  t2, save_ball_y
83     li    t2, 0            # clamp to top edge
84     neg   t4, t4           # reverse direction (bounce down)
85
86 save_ball_y:
87     sw    t2, 0(t1)        # store new y
88     sw    t4, 0(t3)        # store new dy
89
90 # --- PADDLE COLLISION CHECK ---
91     la    t0, ball_x
92     lw    a0, 0(t0)        # a0 = ball x
93     la    t0, ball_y
94     lw    a1, 0(t0)        # a1 = ball y
95     li    a2, 8            # a2 = ball width
96     li    a3, 8            # a3 = ball height
97
98     la    t0, paddle_x
99     lw    a4, 0(t0)        # a4 = paddle x
100    li    a5, 188          # a5 = paddle y
101    li    a6, 64           # a6 = paddle width
102    li    a7, 8            # a7 = paddle height
103
104    call   prg32_sprite_hitbox # a0 = non-zero if overlapping
105    beqz   a0, no_overlap     # no overlap: skip the bounce
106
107    # overlap found!

```



```

108     # play a sound
109     li    a0, 440                # 440 Hz: a short beep on start
110     li    a1, 60                # 60 ms duration
111     call  prg32_audio_beep      # silent on QEMU, audible on the
    board
112
113     # reverse direction
114     la    t0, ball_dy
115     lw    t1, 0(t0)
116     blez  t1, no_overlap        # only bounce if ball is moving
    down (t1 > 0)
117     neg   t1, t1                # Reverse dy natively
118     sw    t1, 0(t0)
119
120 no_overlap:
121     lw    ra, 12(sp)
122     addi  sp, sp, 16
123     ret
124
125 pong_skeleton_draw:
126     addi  sp, sp, -16
127     sw    ra, 12(sp)
128
129     li    a0, 0                # RGB565 black
130     call  prg32_gfx_clear      # black background
131
132     la    t0, paddle_x        # draw the paddle
133     lw    a0, 0(t0)
134     li    a1, 188
135     li    a2, 64
136     li    a3, 8
137     li    a4, 65535            # RGB565 white
138     call  prg32_gfx_rect
139
140     la    t0, ball_x          # draw the ball
141     lw    a0, 0(t0)
142     la    t0, ball_y
143     lw    a1, 0(t0)
144     li    a2, 8
145     li    a3, 8
146     li    a4, 65504            # yellow
147     call  prg32_gfx_rect
148
149     lw    ra, 12(sp)
150     addi  sp, sp, 16
151     ret

```

Listing 6.5. pong_skeleton.S

The draw of the complete game. Compare it line for line with pong_graphics_draw in the repository.

A brief stylistic note: earlier chapters tended to use simple numeric labels for branch targets; in this chapter we switch to descriptive, named labels for different sections of control flow. This is a readability improvement only; it does not change program behaviour but makes the structure and intent of each labelled section easier to follow.

Both targets. Develop and watch this on the QEMU virtual screen, where the ball bounces and the paddle responds to injected input. Then build the identical source for the ESP32-C6 to play it with a real joystick and hear the beep. Because QEMU’s defaults disable the physical GPIO buttons, the paddle may sit still in the emulator unless input is injected; the ball still moves, so the render path is fully verified on the desktop ([Montella et al., 2026e](#)).

6.6 Step-by-step: how this game was built

It is worth making explicit the pedagogical order in which a game like this is constructed, because it is the order the platform recommends and the order in which the example games in `examples/games` were themselves written ([Montella et al., 2026i,l](#)).

First, write `init`, `update`, and `draw` that only return, and confirm the framework runs them. Second, add one `.data` variable. Third, read input in `update` and move the objects, then add clamping. Fourth, add the collision test. Only then, fifth, add sound on the collision event. Each step is buildable and observable on its own; if a step breaks, the bug is in the few lines you just added, not hidden in a wall of untested code.

Try It Yourself

Build the game incrementally in exactly these five steps, rebuilding and running after each. Keep a short log: after each step, write one line saying what you added and what you saw. This log is not busywork: it is the written form of the “change one thing, observe, record” method, and it will make the difference between a frustrating afternoon and a productive one ([Montella et al., 2026f](#)).

Checkpoint

You can now clear the frame, draw rectangles from stored state, move the ball with velocity and bounces, and detect collisions with one call. You have, in other words, built a complete graphical game in RISC-V assembly that runs on both targets. Chapter 7 turns this into a capstone you extend and package as a cartridge.

CHAPTER 7

Capstone: Extending and Shipping an Assembly Game

A program that runs only when embedded in the firmware is a lab exercise; a program packaged as a cartridge is a game you can hand to someone. This closing chapter of the assembly track does two things. First, it extends the Pong of Chapter 6 into a fuller game by reading, not rewriting, the platform’s richer examples. Second, it ships your work: it builds your source into a `.prg32` cartridge and deploys it to both QEMU and the physical board without reflashing the firmware (Montella et al., 2026i,b).

7.1 Reading is a skill: Breakout as Pong with bricks

You will spend far more of your career reading code than writing it, and the platform’s example games are designed to be read. Once you have written Pong, the Breakout example is legible at a glance: it is Pong with an extra piece of state, a row of bricks, and one extra rule: removing a brick when the ball hits it (Montella et al., 2026f). The Breakout source even stores its bricks as a small bitmask, the same bitmask idea you learned for the input in Chapter 5, now repurposed so that each set bit means “this brick is still present.”

Concept

A bitmask is not only for input. Breakout packs a row of bricks into the bits of a single word: bit i set means brick i is intact, clear means it has been knocked out. Testing a brick is `andi`; clearing one is a bitwise AND with the complement of its bit. The same instruction you used to read a button now manages a row of bricks.

Try It Yourself

Open `examples/games/breakout/graphics/game.S` and find the line that declares the brick state (it is a single `.word` near the top, holding a small mask). Then find, in `update`, where a brick is removed. Write two sentences: one naming the register that holds the brick mask, and one explaining which bitwise operation clears a brick. You are now reading assembly the way a maintainer does (Montella et al., 2026f).

7.2 Extending your own game

Before packaging, extend the Pong you built in Chapter 6 with one new feature of your choosing. The platform’s suggested progression, one moving object, one collision, one score counter, then sound, means the natural next features are a score that increases on a successful bounce, a second collision surface, or a beep on the event (Montella et al., 2026i). Add exactly one, build it, and confirm it on QEMU. Resist the urge to add three features at once; the whole method of this book is that a single change is easy to verify and a triple change is not.

A score is the gentlest extension and reinforces the load–modify–store cycle from Chapter 4. In `update`, when the collision test succeeds, read the score word, add one, and store it back. In `draw`, render it with `prg32_gfx_text8`, whose arguments are the position, the string, and a foreground and background colour. Because turning a number into a string is fiddly in pure assembly, a good first version simply draws a fixed label such as "SCORE" and proves the text path works; printing the digits can come later with the console’s `prg32_console_hex32` helper, which displays a value in hexadecimal without any string conversion on your part (Montella et al., 2026f).

Watch Out

Increment the score on the *event*, not every frame. The collision test is true for as long as the rectangles overlap, which may be several frames, so a naive “add one whenever overlapping” counts a single hit many times. The robust pattern is to act on the *transition* into overlap; a simple version remembers whether the ball was overlapping last frame and scores only when it newly overlaps. This is the same “act on the event” discipline as the beep warning in Chapter 2.

7.3 Building a cartridge

When your firmware has been flashed once and your game source is ready, the `prg32_game.py` tool compiles and packages it into a `.prg32` cartridge. The tool needs your source file, the firmware ELF so it can resolve the framework symbols your game calls, the entry-point prefix that names your three functions, a human-readable name, and an output path (Montella et al., 2026i).

Concept

A cartridge bundles your three compiled functions with a small header that the firmware reads to find them. The `-entry-prefix` you pass is how the tool knows which symbols are your `init`, `update`, and `draw`; it must exactly match the prefix in your `.global` directives. A mismatch here is a common build-time error, and the message names the symbol it could not find.

7.4 When it goes wrong

The platform ships a self-check command that verifies your toolchain prerequisites and the partition layout before you spend time chasing an error (Montella et al., 2026f). If a cartridge upload to QEMU fails, the usual cause is a missing or invalid QEMU flash image file; run QEMU once to create it, then stage again. If the board shows a black screen, the usual

cause is targeting the wrong firmware variant, as warned in Chapter 2. These are not exotic failures; they are the everyday friction of embedded work, and learning to read the error and apply the known fix is itself part of the curriculum.

Checkpoint

The assembly track is complete. You can read a register-level game written by someone else; you can extend a game by one well-tested feature; and you can package your work as a cartridge and deploy it to both the emulator and real hardware. You have, end to end, written and shipped a native RISC-V program. Appendix C contains the full annotated source of the games referenced here, and Appendix A is your reference for every framework call you have used.

Part III

The Computer Programming Track: The C Language

CHAPTER 8

First Contact: Functions, Variables, and the Game Loop in C

This chapter begins the computer programming track. It is independent of the assembly track: you need not have read Part II, only the shared foundation in Part I. Here you will write a complete retro game in C, running on both QEMU and the ESP32-C6. The same three-function shape from Chapter 2 organises the work, but where assembly exposed registers and the stack, C lets you write the game’s logic almost as plainly as you would describe it in words ([Montella et al., 2026k](#)).

8.1 Why C, and why the same platform

C occupies a deliberate middle ground. It is a high-level language, you write functions, name variables, and use `if` and `while`, yet it stays close enough to the machine that nothing important is hidden. A C variable is a named box in memory; a C function compiles to the very calling convention you would write by hand in assembly. By learning C on the same platform, the same display, and the same two targets used for the architecture course, you keep your attention on the language and the game logic rather than on a new toolchain ([Montella et al., 2026h](#)). If you later read Part IV, you will see your C compiled into exactly the same assembly idioms used in Part II.

Concept

The platform runs C and assembly games through the identical `init`, `update`, `draw` contract and the identical framework interface. The only thing that changes between the two tracks is the language you express the logic in. This is why the platform can serve both a computer architecture course and a computer programming course from one codebase ([Montella et al., 2026h](#)).

8.2 The three functions, in C

In C, the three entry points are ordinary functions returning `void`, named with your game’s prefix. The framework calls `<prefix>_init` once and then `<prefix>_update` and `<prefix>_draw` once per frame, exactly as in the assembly track ([Montella et al., 2026k](#)). To use the framework, include one header.

```

1  #include "prg32.h"
2
3  void hello_world_c_init(void) {
4  }
5
6  void hello_world_c_update(void) {
7  }
8
9  void hello_world_c_draw(void) {
10 }

```

Listing 8.1. `hello_world_c.c`: The smallest C game. Three functions that do nothing yet: the C counterpart of the assembly skeleton in Chapter 4.

The single `#include "prg32.h"` gives you every framework function and every named constant: the button bits, the colours, and the screen dimensions that you met in Chapter 2. You never declare these yourself; the header does it for you. For a complete list, you can check Appendix A.

Checkpoint

Build this skeleton for QEMU and confirm it runs without crashing and shows a blank screen. As in the assembly track, an empty screen is the correct first result: the framework has found and called your three functions.

8.3 State: variables that persist between frames

A game remembers things between frames. In assembly that meant the `.data` section; in C it means variables declared outside the functions, at file scope, so they keep their values from one frame to the next. The platform's C Pong declares its paddle and ball state exactly this way (Montella et al., 2026f).

```

1  #include "prg32.h"
2
3  static int paddle_x;
4  static int ball_x;
5  static int ball_y;
6  static int ball_dx;      /* horizontal velocity */
7  static int ball_dy;      /* vertical velocity */

```

Listing 8.2. `pong_c_state.c`

Game state as file-scope variables. Declared outside the functions, they persist across frames. `static` keeps them private to this file.

The `init` function is where you can set the starting values of the variables. In the pong example, this is where the initial position and velocities are set.

```

1  void pong_c_init(void) {
2      paddle_x = 128;
3      ball_x = 156;

```

```

4     ball_y = 96;
5     ball_dx = 2;
6     ball_dy = 1;
7 }

```

Listing 8.3. pong_c_init.c

Initialising state in `init`. This runs once, before the first frame.

Concept

File-scope variables in C play the role that labelled words in `.data` play in assembly: they are the game's persistent state. `init` gives them their opening values; `update` changes them each frame; `draw` reads them to render. Keeping state in named variables, rather than recomputing it, is what makes a game a game rather than a static picture.

8.4 Reading input: the bitmask in C

Input works just as it did in Chapter 2: one call returns a bitmask, and you test a button with bitwise AND against a named constant. In C the call returns a value you store in a variable, and the test is an `if`.

```

1 void pong_c_update(void) {
2     uint32_t input = prg32_input_read();
3     if (input & PRG32_BTN_LEFT) {
4         paddle_x -= 3;
5     }
6     if (input & PRG32_BTN_RIGHT) {
7         paddle_x += 3;
8     }
9     if (paddle_x < 0) { paddle_x = 0; }
10    if (paddle_x > 256) { paddle_x = 256; } /* 320 - 64-wide
    paddle */
11 }

```

Listing 8.4. pong_c_update.c

Reading input and moving the paddle. Note the bitwise `&`, not `==`, exactly as warned in Chapter 2.

Watch Out

Use `input & PRG32_BTN_LEFT`, with a single ampersand for bitwise AND, not `input == PRG32_BTN_LEFT`. The mask may hold several buttons at once; the bitwise test asks “is the left bit set, regardless of the others,” which is what you want. This is the same warning as in the assembly track, and it catches beginners in both languages ([Montella et al., 2026k](#)).

The two clamps at the end are the boundary check from Chapter 5, written in C. The maximum is the screen width minus the paddle's own width, so the paddle's right edge never leaves the viewport. In C the clamp is two short `if` statements; the logic is identical to the assembly version, only more compact.

8.5 The ball: velocity and bounce, in C

The ball moves by adding its velocity to its position each frame and bounces by negating its velocity at the edges: the same rule as in Chapter 6, now in three lines of C ([Montella et al., 2026f](#)).

```

1 ball_x += ball_dx;
2 ball_y += ball_dy;
3 if (ball_x < 0 || ball_x > 312) {    /* 320 - 8-wide ball */
4     ball_dx = -ball_dx;              /* bounce horizontally */
5 }
6 if (ball_y < 0 || ball_y > 192) {
7     ball_dy = -ball_dy;              /* bounce vertically */
8 }
```

Listing 8.5. pong_c_ball_motion.c

Ball motion and edge bounce. Adding velocity moves the ball; negating it at the boundary turns it around.

8.6 Collision: one call, one decision

To bounce the ball off the paddle, ask the framework whether their rectangles overlap. `prg32_sprite_hitbox` takes the position and size of two rectangles and returns non-zero when they touch, so collision is a single call inside an `if` ([Montella et al., 2026f](#)).

```

1 if (prg32_sprite_hitbox(
2     ball_x, ball_y, 8, 8,
3     paddle_x, 188, 64, 8)) {
4     ball_dy = -1;                    /* send the ball back upward */
5 }
```

Listing 8.6. pong_c_paddle_collision.c

Bouncing the ball off the paddle. The hitbox test returns non-zero on overlap.

Concept

Collision detection in C is one function call and one `if`. The framework does the geometric work of testing overlap on both axes; you decide what happens when the answer is yes. This separation, the engine answers “did they touch,” your code decides “so what”, is a pattern you will meet throughout game programming.

8.7 Drawing the frame

The `draw` function clears the viewport and renders each object as a coloured rectangle, using the named colour constants from the header. The `draw` function reads the state variables but does not change them; all logic stays in `update`. We do this to follow the phase separation from Chapter 2. ([Montella et al., 2026k](#)).

```

1 void pong_c_draw(void) {
```

```

2   prg32_gfx_clear(PRG32_COLOR_BLACK);
3   prg32_gfx_rect(paddle_x, 188, 64, 8, PRG32_COLOR_WHITE);
4   prg32_gfx_rect(ball_x, ball_y, 8, 8, PRG32_COLOR_YELLOW);
5   prg32_gfx_text8(8, 96, "PONG C", PRG32_COLOR_GREEN,
6   PRG32_COLOR_BLACK);
7 }

```

Listing 8.7. pong_c_draw.c

Rendering the frame. draw only reads state and puts pixels; it changes nothing.

Now we can look at the full Pong example, built in C. If you followed the assembly course, you can compare it line by line with the previous version.

```

1  #include "prg32.h"
2
3  /* Persistent game state, equivalent to the .data section in the
4     assembly version. */
5  static int paddle_x;
6  static int ball_x;
7  static int ball_y;
8  static int ball_dx; /* horizontal velocity */
9  static int ball_dy; /* vertical velocity */
10
11 void pong_skeleton_init(void) {
12     /* Same starting values as the assembly data section. */
13     paddle_x = 140;
14     ball_x = 80;
15     ball_y = 80;
16     ball_dx = 2;
17     ball_dy = 2;
18 }
19
20 void pong_skeleton_update(void) {
21     uint32_t input = prg32_input_read();
22
23     /* --- PADDLE INPUT AND X MOVEMENT --- */
24     if (input & PRG32_BTN_LEFT) {
25         paddle_x -= 3;
26     }
27     if (input & PRG32_BTN_RIGHT) {
28         paddle_x += 3;
29     }
30
31     /* Clamp the paddle to [0, 256] */
32     if (paddle_x < 0) {
33         paddle_x = 0;
34     } else if (paddle_x > 256) {
35         paddle_x = 256;
36     }
37
38     /* --- BALL X MOVEMENT --- */
39     ball_x += ball_dx;
40     if (ball_x < 0) {
41         ball_x = 0;
42         ball_dx = -ball_dx;

```

```

42     } else if (ball_x > 312) {
43         ball_x = 312;
44         ball_dx = -ball_dx;
45     }
46
47     /* --- BALL Y MOVEMENT --- */
48     ball_y += ball_dy;
49     if (ball_y < 0) {
50         ball_y = 0;
51         ball_dy = -ball_dy;
52     }
53
54     /* --- PADDLE COLLISION CHECK --- */
55     if (prg32_sprite_hitbox(
56         ball_x, ball_y, 8, 8,
57         paddle_x, 188, 64, 8)) {
58         /* Match the assembly: beep and only reverse dy when the
59         ball is moving down. */
60         prg32_audio_beep(440, 60);
61         if (ball_dy > 0) {
62             ball_dy = -ball_dy;
63         }
64     }
65
66 void pong_skeleton_draw(void) {
67     prg32_gfx_clear(PRG32_COLOR_BLACK);
68     prg32_gfx_rect(paddle_x, 188, 64, 8, PRG32_COLOR_WHITE);
69     prg32_gfx_rect(ball_x, ball_y, 8, 8, PRG32_COLOR_YELLOW);
70 }

```

Listing 8.8. pong_skeleton.c

Rendering the frame. `draw` only reads state and puts pixels; it changes nothing.

Both targets. This complete C Pong runs unchanged on QEMU and the ESP32-C6. On the emulator you watch the ball bounce and verify the layout; on the board you play it with a real joystick.

Try It Yourself

Change the paddle colour from `PRG32_COLOR_WHITE` to `PRG32_COLOR_CYAN` and the ball size from 8 to 12, then rebuild. Both changes should be visible immediately. Next, the platform’s break-and-fix exercise: delete the `paddle_x > 256` clamp, rebuild, hold `RIGHT` until the paddle leaves the screen, then restore the clamp and explain in one sentence which condition protects the boundary (Montella et al., 2026k).

Checkpoint

You can now write a complete C game: persistent state in file-scope variables, input read as a bitmask and tested with `&`, motion by velocity with boundary clamps, bounce by negation, collision by one call, and rendering kept separate in `draw`. Chapter 9 scales these ideas up with the C features, arrays and structs, that let a game hold many objects and a whole world.

CHAPTER 9

A Bigger World: Arrays, Structs, and Tile Maps

The Pong of Chapter 8 held a handful of separate variables. Real games hold many things at once: rows of bricks, swarms of enemies, a whole landscape of ground and platforms. This chapter introduces the two C features that make such games manageable: the *array*, which holds many values of the same kind, and the *struct*, which groups related values into one named thing, and applies them to a tile-based platform game built on the platform's own example (Montella et al., 2026f,k).

9.1 Arrays: many values under one name

An array is a numbered row of values of the same type. Where Pong needed one paddle position, Breakout needs a position, or simply a present/absent flag, for each of many bricks, and writing a separate variable for each would be absurd. An array lets you store them together and reach any one by its index.

```
1 static uint8_t bricks[10];    /* bricks[i] == 1 means brick i is
   intact */
2
3 static void reset_bricks(void) {
4     for (int i = 0; i < 10; ++i) {
5         bricks[i] = 1;        /* every brick starts present */
6     }
7 }
```

Listing 9.1. An array of brick flags. Index *i* holds whether brick *i* is still present.

Concept

An array stores many values of one type in a numbered sequence; `bricks[i]` is the value at index *i*, counting from zero. A `for` loop visits every element in turn. Together, the array and the loop replace dozens of separate variables and the repetitive code that would manage them.

The framework itself uses small arrays to describe artwork. An 8-by-8 tile: a patch of ground,

a coin or a hazard, is stored as an array of eight bytes, one per row, where each bit is a pixel. The platformer example defines its tiles exactly this way (Montella et al., 2026f).

```
1 static const uint8_t tile_coin[8] = {
2     0x18, 0x3c, 0x66, 0x5a, 0x5a, 0x66, 0x3c, 0x18,
3 };
```

Listing 9.2. `coin_tile.c`

An 8-by-8 tile as an array of eight bytes. Each byte is one row; each bit is one pixel. This is the platformer’s coin tile.

Historical Note

Reading the bits of `tile_coin` as ones and zeros, top to bottom, sketches a small round coin: the rows are narrow at top and bottom and wide in the middle. Defining graphics as bit patterns in arrays is exactly how the home computers of the retro era stored their character sets and sprites, with artists working in hexadecimal on grid paper. The technique is old, compact, and still perfectly legible.

9.2 Structs: grouping what belongs together

A moving character has several attributes that always travel together: its position, its velocity, its size, its state. Keeping them in separate arrays that must be kept in step is error-prone. A *struct* bundles them into a single named value, so the whole character is one thing you can pass around. The framework defines such a struct for platform-game characters, `prg32_platform_actor_t`. This is used in the platformer example, which can be found in `examples/games/platformer/c/game.c` (Montella et al., 2026f).

```
1 typedef struct {
2     int x;           /* position */
3     int y;
4     int vx;          /* velocity */
5     int vy;
6     uint16_t w;      /* size */
7     uint16_t h;
8     uint16_t state; /* collision/status flags */
9     uint8_t layer;
10    uint8_t reserved;
11 } prg32_platform_actor_t;
```

Listing 9.3. The framework’s actor struct (from `prg32.h`). Position, velocity, size, and state, grouped into one value.

Concept

A struct groups related values into one named type. With `prg32_platform_actor_t` `player`; you create a single `player` whose position is `player.x`, velocity `player.vx`, and so on. Passing one struct is cleaner and safer than passing six loose variables, and it scales: an array of structs is a crowd of characters.

9.3 A tile map: the world as a grid

A platform game's world is too large to draw as individual rectangles. Instead it is a *tile map*: a grid in which each cell names a tile to draw there. The framework provides a two-layer playfield for this, and you fill it by placing tile identifiers at grid coordinates with `prg32_playfield_put` (Montella et al., 2026f). The platformer example builds its landscape with small helpers that draw runs and columns of tiles: ordinary loops over the grid.

```

1 static void put_run(uint8_t layer, uint8_t x0, uint8_t x1,
2                     uint8_t y, uint8_t tile) {
3     for (uint8_t x = x0; x <= x1 && x < PRG32_PLAYFIELD_COLS; ++x
4     ) {
5         prg32_playfield_put(layer, x, y, tile);
6     }
7 }
```

Listing 9.4. Filling a run of ground tiles with a loop. The platformer uses helpers like this to lay out its world.

Concept

A tile map represents a large world compactly: instead of remembering every pixel, it remembers, for each grid cell, which small reusable tile belongs there. Drawing the world becomes drawing the grid; building the world becomes filling the grid with loops. This is how side-scrolling games fit sprawling levels into tiny memory.

9.4 Behaviour from tile flags

A tile is not only a picture; it can carry *behaviour*. The framework lets you attach flags to a tile identifier, solid, platform, hazard, collectable, and then the physics helpers know how a character should interact with it. You declare, for instance, that tile 2 is solid, and from then on a character cannot walk through it (Montella et al., 2026f).

```

1 prg32_platform_tile_flags(2, PRG32_TILE_FLAG_SOLID);    /* ground
   is solid */
2 prg32_platform_tile_flags(5, PRG32_TILE_FLAG_HAZARD);  /* spikes
   hurt */
3 prg32_platform_tile_flags(6, PRG32_TILE_FLAG_COLLECT); /* coins
   collect */
```

Listing 9.5. Giving tiles behaviour. Marking a tile solid makes the physics helper treat it as ground or wall.

9.5 Putting it together: an actor in a world

With an actor struct, a tile map, and tile flags, a platform game's per-frame logic becomes remarkably short. The framework's `prg32_platform_actor_step` reads the input, applies gravity, moves the actor, and resolves collisions against the solid tiles, updating the struct's `state` flags so your code can react to what happened (Montella et al., 2026f,k).

```

1 static prg32_platform_actor_t player;
2
3 void platformer_c_init(void) {
4     fill_world(); /* build the
5                   tile map */
6     prg32_platform_tile_flags(2, PRG32_TILE_FLAG_SOLID);
7     prg32_platform_actor_init(&player, 1, 32, 120, 8, 8);
8 }
9
10 void platformer_c_update(void) {
11     uint32_t input = prg32_input_read();
12     /* move_speed, jump_speed, gravity, max_fall */
13     uint16_t state = prg32_platform_actor_step(&player, input, 2,
14     -7, 1, 5);
15     if (state & PRG32_PLATFORM_HAZARD) {
16         prg32_audio_beep(120, 60); /* hit a
17                                     hazard */
18     }
19     prg32_platform_camera_follow(&player, 64, 48); /* scroll
20                                                         the world */
21 }

```

Listing 9.6. One frame of platform physics. The helper does the movement and collision; your code reads the resulting state.

A single call advances the whole character: the joystick moves it, gravity pulls it down, the solid tiles stop it, and the returned **state** tells you if it touched a hazard. The camera-follow call then scrolls the tile map so the character stays in view. The same behaviour written from first principles would be a page of code; the struct and the helper compress it because they capture exactly the data and the operation the problem needs.

Watch Out

Pass the *address* of the actor, `&player`, to the stepping and follow helpers, not the struct by value. These functions must *change* the actor, its position and state, and in C a function can only change a value the caller can see if it is given that value's address. Forgetting the `&` is a common error; the symptom is an actor that never moves, because the function updated a private copy.

Try It Yourself

Open `examples/games/platformer/c/game.c` and find where it defines the coin tile and where it marks a tile collectable. Add one new platform to the world by calling `put_run` with new coordinates, rebuild for QEMU, and confirm your platform appears. Then change the `gravity` argument of `prg32_platform_actor_step` from 1 to 2 and describe how the jump feels different. One change, observed and recorded ([Montella et al., 2026f](#)).

Checkpoint

You can now hold many values in arrays, group related values in structs, represent a world as a tile map filled by loops, and give tiles behaviour with flags. These are the data-structuring ideas a programming course exists to teach, learned here on a game you can see and play. Chapter 10 brings them together into a capstone you extend and ship as a cartridge.

CHAPTER 10

Capstone: Extending and Shipping a C Game

This chapter closes the C track the way Chapter 7 closed the assembly track: by extending a game and shipping it. You will embed a C game in the firmware for quick iteration, build it into a .prg32 cartridge, and deploy it to both QEMU and the board. Along the way you will meet two ideas that recur in serious systems programming: reusing helper functions, and doing arithmetic without a floating-point unit, both grounded in the platform's richer C examples ([Montella et al., 2026k,f](#)).

10.1 Embedding a C game in the firmware

For fast iteration during development, the platform lets you compile a C game directly into the firmware instead of packaging a cartridge each time. You add the game's source to the firmware's build list and call its three functions from the main loop ([Montella et al., 2026k](#)). This is a temporary lab arrangement, restored afterwards, but it shortens the edit-build-run cycle while you are still changing the game often.

```
1  #include "prg32.h"
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4
5  void pong_c_init(void);
6  void pong_c_update(void);
7  void pong_c_draw(void);
8
9  void app_main(void) {
10     prg32_init();
11     pong_c_init();
12     while (1) {
13         pong_c_update();
14         pong_c_draw();
15         prg32_gfx_present();           /* show the finished
frame */
16         vTaskDelay(pdMS_TO_TICKS(33)); /* ~30 frames per second
*/
17     }
18 }
```

Listing 10.1. Calling a C game’s three functions from the firmware’s main loop. `init` once, then `update` and `draw` each frame, with a `present` and a short delay.

Concept

The frame loop you met as a diagram in Chapter 2 is, in the firmware, a literal `while` (1) that calls `update`, then `draw`, then `prg32_gfx_present` to display the frame, then waits briefly so the game runs at a steady rate. Embedding your game here lets you see the loop you have been writing the body of all along.

Watch Out

The embedding is temporary. The platform’s tutorial is explicit that you should restore the firmware’s default main loop and build list after the lab, so the resident firmware returns to loading cartridges normally (Montella et al., 2026k). Keep a clean copy, or use version control (Appendix D) so restoring is one command.

10.2 Reusing helpers and reading richer examples

Once your game grows, repeated code is a liability. C’s answer is the helper function: a named, reusable piece of logic you write once and call many times. You already saw the platformer’s `put_run` and `put_column` helpers in Chapter 9; they exist precisely so the world-building code is not a wall of near-identical lines. When you extend your own game, watch for three or more lines that recur with small variations, and lift them into a helper.

The platform’s raycaster example is the next step for a class ready for more (Montella et al., 2026h). It renders a Doom-style corridor, a convincing pseudo-3-D view, on the same RISC-V runtime used for everything else. What makes it instructive is not the 3-D effect but *how* it computes that effect without a floating-point unit.

10.3 Arithmetic without floating point: fixed point

Small microcontrollers often have no hardware for fractional numbers, yet a raycaster needs fractions: half a step, a third of an angle. The classic solution, used by the platform’s raycaster, is *fixed-point arithmetic*: represent a fractional value as an integer that has been scaled up by a fixed factor, do the arithmetic in integers, then scale back down (Montella et al., 2026k,f).

```
1 /* dir_q8 holds direction components scaled by 256 (called Q8
   fixed point). */
2 int hx = player_x + (dir_q8[angle][0] * dist) / 256;
3 int hy = player_y + (dir_q8[angle][1] * dist) / 256;
```

Listing 10.2. Fixed-point in the raycaster. Direction vectors are stored scaled by 256; multiplying and then dividing by 256 keeps the fraction.

Concept

Fixed-point arithmetic stores a fractional number as an integer scaled by a fixed factor: here, 256. A value of 1.5 is stored as 384, because $1.5 \times 256 = 384$. Multiplications are done in integers and then divided by the scale to bring the result back. This lets a processor with no floating-point unit compute smooth motion and angles using only integer instructions, a technique as old as real-time graphics and still used in embedded systems today.

Historical Note

The notation “Q8” in the example names the format: eight bits of fraction, so a scale of $2^8 = 256$. Fixed-point was the only practical way to do fast fractional maths on the home computers of the retro era, and the raycaster’s corridor is a direct descendant of the techniques that made early 3-D games run on machines with no floating-point hardware at all. Learning it here is learning a genuine piece of computing history that remains useful.

10.4 Building and shipping a C cartridge

For updated instructions on building and uploading cartridges (both for QEMU and for the physical board), see Chapter 3.

Both targets. A C cartridge and an assembly cartridge are the same kind of file and run through the same loader; the framework neither knows nor cares which language produced the three functions inside (Montella et al., 2026h). This is the deepest demonstration of the platform’s design: two courses, two languages, one runtime, two targets, one cartridge format.

Try It Yourself

Take the C Pong you wrote in Chapter 8, add a score that increments when the ball bounces off the paddle, guarding against counting one hit many times, as warned in Chapter 7, and display it with `prg32_gfx_text8`. Build it as a cartridge for QEMU and run it. Then, if you have a board, upload the same cartridge over Wi-Fi and play it. You will have written, extended, packaged, and shipped a complete C game on real RISC-V hardware (Montella et al., 2026f).

Checkpoint

The C track is complete. You can embed a game for fast iteration, factor repeated logic into helpers, reason about fixed-point arithmetic, and build and deploy a C cartridge to both targets. With both tracks behind you (or your own track behind you), Chapter 11 offers a short, illuminating reunion: the same game, in both languages, side by side.

Part IV

Bringing It Together

CHAPTER 11

Assembly Meets C: One Game, Two Altitudes

This short chapter is the book’s reunion. If you have read both tracks, it will crystallise something you have half-seen already; if you have read only one, it is an invitation to glance over the wall at the other side. The claim is simple and, once shown, hard to unseen: the assembly game and the C game are the same game, viewed at two altitudes, and the compiler is the translator between them (Montella et al., 2026h).

11.1 The same logic, two languages

Consider the single most common operation in any of our games: move an object left when the left button is pressed. In C it is three lines you could read aloud as English.

```
1 uint32_t input = prg32_input_read();
2 if (input & PRG32_BTN_LEFT) {
3     paddle_x -= 3;
4 }
```

Listing 11.1. Move left, in C. Read input, test the bit, change the position.

In assembly it is the same idea spelled out in the machine’s own terms: call, copy the result, mask the bit, branch if clear, subtract, and because `paddle_x` lives in memory, load before and store after.

```
1 call prg32_input_read      # input bitmask in a0
2 mv    t0, a0               # keep our copy
3 la    t1, paddle_x
4 lw    t2, 0(t1)            # load paddle_x from memory
5 andi  t3, t0, 1            # isolate the LEFT bit
6 beqz  t3, 1f               # if LEFT is clear, skip
7 addi  t2, t2, -3           # paddle_x -= 3
8 1: sw  t2, 0(t1)           # store paddle_x back
```

Listing 11.2. Move left, in assembly. The same logic, with every step the C compiler would have generated made explicit.

Line up the two listings and the correspondence is exact. The C `if (input & PRG32_BTN_LEFT)` becomes `andi` followed by `beqz`. The C `paddle_x -= 3` becomes a load, an `addi`, and a store, because a C variable in memory must be brought into a register to be changed and written back: the load–modify–store cycle of Chapter 4. Nothing in the assembly is arbitrary; every instruction is doing work the C statement implied.

Concept

A compiler translates each C construct into a fixed pattern of machine instructions: a bitwise test becomes a mask and a branch, an assignment to a variable in memory becomes load–modify–store, a function call becomes the calling convention. C is not magic that the machine somehow understands; it is a notation that a compiler mechanically turns into exactly the assembly you would have written by hand.

11.2 What C hides, and what it does not

The C version is shorter not because it does less but because the compiler writes the bookkeeping for you. It chooses the registers, emits the loads and stores, manages the stack frame, and saves `ra` around calls, all the discipline you practised by hand in Part II. This is C’s bargain: you express intent, the compiler handles the mechanism.

What C does *not* hide, on this platform, is the machine itself. The same display, the same input bitmask, the same RGB565 colours, the same three-function contract, the same two targets. A C variable is still a box in memory; a collision is still a comparison; a frame is still a turn around a loop. The abstractions of C sit directly on the concepts of Part I, which is why a student who has seen both tracks understands C more deeply than one who has only ever used it: they know what the compiler is doing on their behalf.

11.3 A side-by-side map

The platform’s teaching guide recommends exactly this comparison as a classroom activity, using the platformer example, whose assembly and C versions implement the same game (Montella et al., 2026h). Table 11.1 lays the two languages alongside each other for the concepts you have met.

Concept	In assembly (Part II)	In C (Part III)
Persistent state	labelled words in <code>.data</code>	file-scope variables
A grouped object	a block of memory at fixed offsets	a <code>struct</code>
Many objects	an array of words	an array
Read input	<code>call</code> , then <code>andi/beqz</code>	<code>prg32_input_read</code> and <code>if (... & ...)</code>
Change a variable	load, modify, store	<code>x += ...</code>
A decision	a conditional branch	an <code>if</code>
Call the framework	manage <code>ra</code> and the stack	write the call
Move an actor	<code>call</code> <code>prg32_platform_actor_step</code>	the same call, by name

Table 11.1. The same ideas in both languages. The right column is the left column with the bookkeeping delegated to the compiler (Montella et al., 2026h).

Historical Note

That the framework call `prg32_platform_actor_step` appears identically in both columns is the whole point of the platform’s design. The framework is written in C and exposes a standard RISC-V calling convention, so an assembly game and a C game invoke it the same way. The boundary between “your code” and “the framework” is the same boundary in both languages, which is why the two tracks can share a single runtime, a single set of examples, and a single pair of targets ([Montella et al., 2026a,h](#)).

11.4 Where to go next

You have now written native RISC-V games, in assembly, in C, or in both, that run on a real open-architecture processor and on its emulator. The natural next steps are to read the richer examples you have not yet opened: Space Invaders for many moving enemies, Tetris for a falling-piece grid, the raycaster for fixed-point pseudo-3-D, and Wing Commander for two layered playfields ([Montella et al., 2026f](#)). Each is a variation on patterns you already know, and each is written to be read.

Beyond the games, the appendices that follow are a working reference for everything you have used and several things you have not: the complete framework interface, the workflow for measuring a game’s performance like a scientist, full annotated source for the examples, the use of GitHub to manage and submit your work, the construction of the physical board, and the setup of your development environment on whichever operating system you use.

Checkpoint

You should now be able to look at a short C statement and describe, in words, the assembly the compiler will produce for it, and to look at a fragment of RISC-V assembly and recognise the C construct it implements. That two-way fluency, between the notation and the machine, is the deepest thing a first year of computing can give you, and it is what this book set out to teach.

Part V

Appendices

CHAPTER A

The prg32 Runtime Interface

This appendix is the working reference for the PRG32 framework calls used throughout the book, together with the calling convention, the cartridge tool, and the constants you will reach for most often. It catalogues the public interface declared in `components/prg32/include/prg32.h` and documented in the platform’s ABI notes (Montella et al., 2026f,a). It is organised by task rather than alphabetically, so that you can find a call by what you are trying to do.

A.1 The calling convention

Every framework function follows the standard RISC-V calling convention, which is what allows assembly games and C games to call the same functions (Montella et al., 2026a). Arguments are passed in `a0–a7` in order; the return value comes back in `a0`. Temporaries `t0–t6` may be destroyed by a call; saved registers `s0–s11` are preserved across one. The return address is in `ra` and must be saved before a nested call; the stack pointer `sp` stays 16-byte aligned.

Register	Role	Preserved across a call?
<code>a0–a7</code>	arguments / return value	no
<code>t0–t6</code>	temporaries	no
<code>s0–s11</code>	saved values	yes
<code>ra</code>	return address	no (save it yourself)
<code>sp</code>	stack pointer	yes (16-byte aligned)

Table A.1. The RISC-V register convention as PRG32 uses it (Montella et al., 2026a).

A.2 Lifecycle and timing

Function	Purpose
<code>prg32_init()</code>	initialise the runtime; call once before the loop
<code>prg32_ticks_ms()</code>	milliseconds since start, for timing and animation
<code>prg32_gfx_present()</code>	display the frame you have drawn

Table A.2. Lifecycle and timing calls.

A.3 Input

All buttons arrive as a single bitmask. Player two occupies the same layout shifted into higher bits (PRG32_P2_*) ([Montella et al., 2026f](#)).

Function	Purpose
<code>prg32_input_read()</code>	full player 1 / player 2 bitmask
<code>prg32_input_read_player(p)</code>	one player's bits, normalised to the low bits
<code>prg32_input_read_menu()</code>	both joysticks merged, for menu navigation
<code>prg32_input_wait_released(mask)</code>	block until the selected bits are released

Table A.3. Input calls.

Constant	Bit	Constant	Bit
PRG32_BTN_LEFT	0x01	PRG32_BTN_A	0x10
PRG32_BTN_RIGHT	0x02	PRG32_BTN_B	0x20
PRG32_BTN_UP	0x04	PRG32_BTN_SELECT	0x40
PRG32_BTN_DOWN	0x08	(PRG32_BTN_START alias)	0x40

Table A.4. Player-one button bits ([Montella et al., 2026f](#)).

A.4 Graphics

The viewport is 320 by 200 within a 320-by-240 display; colours are RGB565 ([Montella et al., 2026f](#)).

Function	Purpose
<code>prg32_gfx_clear(color)</code>	fill the viewport with one colour
<code>prg32_gfx_pixel(x,y,color)</code>	set one pixel
<code>prg32_gfx_rect(x,y,w,h,color)</code>	filled rectangle
<code>prg32_gfx_text8(x,y,s,fg,bg)</code>	draw a text string
<code>prg32_gfx_present()</code>	present the finished frame
<code>prg32_gfx_lock() / _unlock()</code>	enter / leave the graphics critical section

Table A.5. Core graphics calls.

Colour	RGB565	Colour	RGB565
PRG32_COLOR_BLACK	0x0000	PRG32_COLOR_YELLOW	0xffe0
PRG32_COLOR_WHITE	0xffff	PRG32_COLOR_CYAN	0x07ff
PRG32_COLOR_RED	0xf800	PRG32_COLOR_MAGENTA	0xf81f
PRG32_COLOR_GREEN	0x07e0	PRG32_COLOR_BLUE	0x001f

Table A.6. Named colour constants ([Montella et al., 2026f](#)).

A.5 Sprites and collision

Function	Purpose
<code>prg32_sprite_hitbox(ax,ay,aw,ah,bx,by,bw,bh)</code>	non-zero if two rectangles overlap
<code>prg32_sprite_anim_frame(now,count,ms)</code>	which animation frame to show now
<code>prg32_sprite_draw_frame(x,y,w,h,frames,frame,transparent)</code>	draw one frame of an animation
<code>prg32_sprite_draw_8x8(x,y,bits,fg,bg)</code>	draw an 8-by-8 bit sprite

Table A.7. Sprite and collision calls ([Montella et al., 2026f](#)).

A.6 Playfields and tiles

The framework provides two scrolling tile layers for larger worlds ([Montella et al., 2026f](#)).

Function	Purpose
<code>prg32_playfield_put(layer,tx,ty,id)</code>	place a tile in the grid
<code>prg32_playfield_get(layer,tx,ty)</code>	read a tile from the grid
<code>prg32_playfield_scroll(layer,x,y)</code>	scroll a layer to a position
<code>prg32_playfield_parallax(layer,x_q8,y_q8)</code>	scroll with fixed-point parallax
<code>prg32_playfield_camera(x,y)</code>	set the shared camera position
<code>prg32_playfield_draw_dual()</code>	draw both layers

Table A.8. Playfield and tile calls.

A.7 Platform-game physics

The actor struct `prg32_platform_actor_t` and its helpers implement gravity, movement, and tile collision ([Montella et al., 2026f](#)).

Function	Purpose
<code>prg32_platform_tile_flags(id,flags)</code>	give a tile behaviour (solid, hazard, ...)
<code>prg32_platform_actor_init(&a,layer,x,y,w,h)</code>	set up an actor
<code>prg32_platform_actor_step(&a,input,move,jump,grav,maxfall)</code>	one frame of physics; returns state flags
<code>prg32_platform_camera_follow(&a,dzx,dzy)</code>	scroll to keep the actor in view

Table A.9. Platform physics calls. Pass the actor by address (&).

Tile flag	Meaning	Actor state	Meaning
..._SOLID	blocks movement	..._ON_GROUND	standing on ground
..._PLATFORM	one-way platform	..._HIT_HEAD	bumped a ceiling
..._HAZARD	harmful tile	..._HAZARD	touched a hazard
..._COLLECT	collectable	..._COLLECT	collected an item

Table A.10. Tile flags (PRG32_TILE_FLAG_*) and actor state bits (PRG32_PLATFORM_*) ([Montella et al., 2026f](#)).

A.8 Console and sound

Function	Purpose
prg32_console_clear()	clear the text console
prg32_console_write(s)	write a string to the console
prg32_console_hex32(v)	write a 32-bit value in hexadecimal
prg32_audio_beep(hz,ms)	play a tone (event sounds only)
prg32_audio_note(midi,ms)	play a MIDI note for a duration

Table A.11. Console and basic sound calls. Sound is audible on the board and silent on QEMU.

A.9 Cartridges and scores

Function	Purpose
prg32_cart_default_slot()	saved default cartridge slot, or -1
prg32_cart_select_default()	load the saved default cartridge
prg32_score_submit(game,player,score)	record a local score
prg32_device_demo_run()	run the built-in device demo

Table A.12. Selected cartridge and score calls ([Montella et al., 2026a](#)).

A.10 The cartridge tool

For detailed instructions on build/upload options and workflows (including upload via Wi-Fi and staging for QEMU), see Chapter 3. The `README.md` and the `docs/` folder contain supporting material and practical examples.

A.11 The HTTP runtime API

When the board’s firmware is running and reachable, it exposes a small HTTP interface useful for tooling and labs ([Montella et al., 2026f](#)).

Endpoint	Returns
GET /api/runtime	firmware version, cartridge state, frame count, last input
GET /api/games	the installed cartridges
POST /api/games?slot=cart0	install a cartridge into a slot
GET /api/screenshot.bmp	the current 320-by-240 framebuffer as a BMP

Table A.13. Selected HTTP runtime endpoints ([Montella et al., 2026f](#)).

CHAPTER B

Measuring Performance Like a Scientist

A game that “feels fast” is an opinion; a game that spends 4.2 milliseconds per frame in `update` is a measurement. This appendix shows how to turn the first into the second using PRG32’s optional performance-metrics pipeline, which records per-frame timing and uploads it to a small local server for analysis (Montella et al., 2026d,g). The workflow is designed to be reproducible, so that a classmate or a reviewer can repeat your experiment and obtain the same result: this is the defining property of a scientific measurement.

B.1 Why measure, and what stays unchanged

Metrics are disabled by default, so ordinary gameplay is never slowed by measurement (Montella et al., 2026d). You enable them only for a profiling lab or a report. When enabled, the runtime begins a measurement *run* when a cartridge starts and ends it when the cartridge is replaced, recording a sample every few frames into an in-memory ring buffer that a background task uploads. Because the recording function merely copies a sample and returns, the game’s own timing is disturbed as little as possible (Montella et al., 2026d).

B.2 What each frame records

A sample captures where a frame’s time went and how healthy the system was (Montella et al., 2026d). The most useful fields for a first report are the three phase timings and the deadline flag.

Field	Meaning
<code>frame</code>	the frame counter
<code>update_us</code>	microseconds spent in your <code>update</code>
<code>draw_us</code>	microseconds spent in your <code>draw</code>
<code>present_us</code>	microseconds spent displaying the frame
<code>frame_us</code>	total work: <code>update</code> + <code>draw</code> + <code>present</code>
<code>fps_x100</code>	active-work frames per second, times 100
<code>heap_free</code>	free memory at that moment
<code>deadline_missed</code>	true when the frame’s work exceeded the 30-FPS budget

Table B.1. Per-frame sample fields (Montella et al., 2026d).

Concept

The frame budget at 30 frames per second is about 33 milliseconds. If **update**, **draw**, and **present** together exceed that, **deadline_missed** is set and the game cannot keep its rate. Profiling is the art of finding which of the three phases is the culprit and reducing it, a habit worth forming early.

B.3 The reproducible workflow

The platform’s scientific-measurement tutorial prescribes a discipline borrowed from experimental science: state the question, freeze the software, measure, and report enough that someone else can repeat it (Montella et al., 2026g).

State a narrow question. Good questions are observable and repeatable: how much frame time does each demo spend in update, draw, and present? Does QEMU produce the same visual output as the board? Which examples stay below the 30-FPS budget? Write the question, one expected result, and one reason it might be wrong, *before* measuring (Montella et al., 2026g).

Freeze the software state. Record the exact commit, the ESP-IDF version, the target (ESP32-C6 hardware or ESP32-C3 QEMU), the display backend, the metrics configuration, and the cartridge build command, so the experiment can be reconstructed (Montella et al., 2026g).

```
git rev-parse HEAD           # the exact source revision
git diff > experiment.patch  # any local changes, saved as a patch
```

Listing B.1. Recording the software state for reproducibility.

Build with metrics enabled. A checked-in configuration profile turns on metrics without disturbing the normal board defaults; you combine the two profiles in one build (Montella et al., 2026d).

```
idf.py -B build-esp32c6-metrics \
  -D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.defaults.
    metrics" \
  set-target esp32c6
idf.py -B build-esp32c6-metrics \
  -D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.defaults.
    metrics" \
  build
```

Listing B.2. Building the board firmware with metrics enabled, using both configuration profiles.

Run the metrics server. A small Flask and SQLite server collects the uploaded samples (Montella et al., 2026d).

```
python3 -m venv .venv
. .venv/bin/activate
python3 -m pip install -r tools/prg32_metrics_server/requirements
.txt
python3 tools/prg32_metrics_server/app.py --host 0.0.0.0 --port
8080
```

Listing B.3. Starting the local metrics server.

Point `CONFIG_PRG32_METRICS_SERVER_URL` at the computer’s address that the board or QEMU can reach, then run your cartridge; the firmware uploads samples in the background as it plays (Montella et al., 2026d).

B.4 Getting results out

The server stores each run and computes summary statistics. You can list runs, inspect one with its statistics, download the raw samples as CSV, or download a ready-made Markdown report (Montella et al., 2026d).

Endpoint	Returns
GET /api/runs	all recorded runs
GET /api/runs/<id>	one run with summary statistics
GET /api/runs/<id>/samples.csv	the raw per-frame samples
GET /api/runs/<id>/report.md	a formatted report

Table B.2. Metrics-server endpoints for retrieving results (Montella et al., 2026d).

For a written report there is an export helper that produces, from one run, a CSV file, a Markdown report, a small $\text{\LaTeX}$ table you can paste into a document, and, when `matplotlib` is installed, PNG plots (Montella et al., 2026d).

```
python3 tools/prg32_metrics_server/export_run.py <run_id> \
--db tools/prg32_metrics_server/metrics.db \
--out metrics_export/<run_id>
```

Listing B.4. Exporting a run into report-ready artefacts.

B.5 A complete profiling exercise

The platform suggests a self-contained experiment that teaches both profiling and the effect of measurement itself on the thing measured (Montella et al., 2026d). Run a cartridge with metrics off and note the frame rate. Enable metrics with a sample every frame and run the same cartridge for thirty seconds. Export the run and compare the update, draw, and present times. Then raise the sample period to one in five frames and repeat. Finally, explain how the measurement overhead and the network conditions changed the numbers.

Both targets. The metrics pipeline works for both the ESP32-C6 and the QEMU target, and the screenshot endpoint returns the same 320-by-240 BMP in either case, which makes “does QEMU match the board?” a question you can answer with evidence rather than impression ([Montella et al., 2026e,g](#)).

Historical Note

The instruction to record one expected result, and one reason it might be wrong before measuring, is at the heart of a honest experiment. A prediction made after seeing the data is no prediction at all. By committing to an expectation first, you turn a casual look at some numbers into a genuine test, and you learn the most from the cases where the machine surprises you.

CHAPTER C

Full Working Examples

This appendix collects complete, working source listings for the games built and referenced in the chapters. Each is reproduced verbatim from the PRG32 repository’s `examples/games` directory, where the sources are deliberately written with a comment on nearly every instruction so that a first-year student can connect each line to its visible effect (Montella et al., 2026f). Read them after the corresponding chapter: the assembly listings reward the reader of Part II, the C listings the reader of Part III, and the side-by-side comparison rewards anyone who has read Chapter 11.

All four programs implement games discussed in the text, and all four are designed to run unchanged on both the QEMU emulator and the physical ESP32-C6, packaged as `.prg32` cartridges by the workflow of Appendix A (Montella et al., 2026e).

C.1 Pong in graphics assembly

This is the complete `pong_graphics` example developed across Chapters 4–6. It exports the three symbols `pong_graphics_init`, `pong_graphics_update`, and `pong_graphics_draw`, keeps its state in `.data`, moves a paddle and a bouncing ball, and renders them with `prg32_gfx_rect`. The optional debug overlay at the end is guarded by a compile-time flag.

```
1  /*
2   * PONG GRAPHICS for PRG32.
3   * This file is intentionally verbose: every instruction is
4   * commented so
5   * first-year students can connect RISC-V assembly to visible
6   * hardware effects.
7   */
8  #include "prg32_config.h"
9  .option norelax
10 .section .data
11 pong_graphics_paddle_x: .word 120      /* Paddle x coordinate in
    pixels. */
12 pong_graphics_paddle_dx: .word 2      /* Paddle horizontal
    velocity. */
13 pong_graphics_ball_x: .word 156      /* Ball x coordinate in
    pixels. */
```

```

12 pong_graphics_ball_y:    .word 96          /* Ball y coordinate in
    pixels. */
13 pong_graphics_ball_dx:  .word 2          /* Ball horizontal
    velocity. */
14 pong_graphics_ball_dy:  .word 1          /* Ball vertical velocity
    . */
15 pong_graphics_score:    .word 0          /* Simple score/counter.
    */
16
17 .section .rodata
18 pong_graphics_title:    .asciz "PONG GRAPHICS\n"
19 pong_graphics_msg:      .asciz "PRG32 RISC-V ASM DEMO\n"
20
21 .section .text
22 .global pong_graphics_init
23 .global pong_graphics_update
24 .global pong_graphics_draw
25
26 pong_graphics_init:
27     /* INIT: runs once when the cartridge is selected. */
28     addi sp, sp, -16          /* Build a tiny stack frame
    before calling C. */
29     sw ra, 12(sp)            /* Save return address because
    call overwrites ra. */
30     call prg32_console_clear /* Clear UART/LCD text console.
    */
31     la a0, pong_graphics_title /* Put address of title string
    into argument a0. */
32     call prg32_console_write /* Print title through
    framework console. */
33     la a0, pong_graphics_msg /* Load second message pointer
    into a0. */
34     call prg32_console_write /* Print explanatory message.
    */
35     li a0, 120              /* Choose a short beep
    frequency/duration argument. */
36     li a1, 30              /* 30 milliseconds: audible but
    not annoying. */
37     call prg32_audio_beep   /* Call C PWM audio helper. */
38     lw ra, 12(sp)          /* Restore return address. */
39     addi sp, sp, 16        /* Destroy stack frame. */
40     ret                    /* Return to the C game loop.
    */
41
42 pong_graphics_update:
43     /*
44     * UPDATE: run autonomous demo motion so QEMU always shows
    activity.
45     * Paddle moves left/right and the ball bounces on all
    viewport edges.
46     */
47     addi sp, sp, -16        /* Keep a normal call-safe
    frame style. */
48     sw ra, 12(sp)          /* Preserve return address. */
49
50     /* Paddle auto motion: x += dx, bounce at [0, 256] for 64-

```

```

pixel paddle. */
51  la t1, pong_graphics_paddle_x
52  lw t2, 0(t1)           /* t2 = paddle x */
53  la t3, pong_graphics_paddle_dx
54  lw t4, 0(t3)           /* t4 = paddle dx */
55  add t2, t2, t4
56  bltz t2, 1f            /* Left wall hit. */
57  li t5, 256             /* 320 - paddle width (64). */
58  ble t2, t5, 2f
59  mv t2, t5               /* Right wall clamp. */
60  neg t4, t4             /* Reverse direction. */
61  j 2f
62  1: li t2, 0             /* Left wall clamp. */
63  neg t4, t4             /* Reverse direction. */
64  2: sw t2, 0(t1)
65  sw t4, 0(t3)
66
67  /* Ball autonomous motion: x/y += dx/dy with edge bounces. */
68  la t1, pong_graphics_ball_x
69  lw t2, 0(t1)           /* t2 = ball x */
70  la t3, pong_graphics_ball_dx
71  lw t4, 0(t3)           /* t4 = ball dx */
72  add t2, t2, t4
73  bltz t2, 3f
74  li t5, 312             /* 320 - ball width (8). */
75  ble t2, t5, 4f
76  mv t2, t5
77  neg t4, t4
78  j 4f
79  3: li t2, 0
80  neg t4, t4
81  4: sw t2, 0(t1)
82  sw t4, 0(t3)
83
84  la t1, pong_graphics_ball_y
85  lw t2, 0(t1)           /* t2 = ball y */
86  la t3, pong_graphics_ball_dy
87  lw t4, 0(t3)           /* t4 = ball dy */
88  add t2, t2, t4
89  bltz t2, 5f
90  li t5, 192             /* 200 - ball height (8). */
91  ble t2, t5, 6f
92  mv t2, t5
93  neg t4, t4
94  j 6f
95  5: li t2, 0
96  neg t4, t4
97  6: sw t2, 0(t1)
98  sw t4, 0(t3)
99
100 la t1, pong_graphics_score /* Load score variable address.
    */
101 lw t2, 0(t1)             /* Load current score/counter.
    */
102 addi t2, t2, 1           /* Increment counter once per
    frame. */

```

```

103     sw t2, 0(t1)                /* Store updated counter. */
104     lw ra, 12(sp)              /* Restore return address. */
105     addi sp, sp, 16            /* Release stack frame. */
106     ret                        /* Return to C scheduler. */
107
108 pong_graphics_draw:
109     /* DRAW: runs every frame and renders the current game state.
110      */
111     addi sp, sp, -16            /* Save state before C drawing
112     calls. */
113     sw ra, 12(sp)              /* Save return address. */
114     li a0, 0                    /* RGB565 black background. */
115     call prg32_gfx_clear       /* Clear 320x200 retro graphics
116     viewport. */
117
118     la t0, pong_graphics_paddle_x /* Paddle x coordinate. */
119     lw a0, 0(t0)
120     li a1, 188                  /* Paddle stays near bottom for
121     visibility. */
122     li a2, 64                    /* Wide paddle is easy to see
123     in QEMU. */
124     li a3, 8
125     li a4, 65535                /* a4 = white color in RGB565.
126     */
127     call prg32_gfx_rect        /* Draw auto-moving paddle. */
128
129     la t0, pong_graphics_ball_x /* Ball x coordinate. */
130     lw a0, 0(t0)
131     la t0, pong_graphics_ball_y /* Ball y coordinate. */
132     lw a1, 0(t0)
133     li a2, 8
134     li a3, 8
135     li a4, 65504                /* Yellow ball. */
136     call prg32_gfx_rect        /* Draw bouncing ball. */
137
138 #if PRG32_DEBUG
139     /*
140     * Optional debug HUD example:
141     * prg32_debug_overlay_draw(1, x, y, input_mask,
142     frame_counter)
143     */
144     call prg32_input_read       /* Read current input for HUD.
145     */
146     mv t1, a0                    /* Keep input mask in t1. */
147     la t0, pong_graphics_score /* Reuse score as a tiny frame
148     counter. */
149     lw t2, 0(t0)
150     li a0, 1                      /* Enable overlay. */
151     la t0, pong_graphics_paddle_x
152     lw a1, 0(t0)                  /* x */
153     li a2, 188                    /* y */
154     mv a3, t1                      /* input mask */
155     mv a4, t2                      /* frame/counter */
156     call prg32_debug_overlay_draw
157 #endif

```



```

150     lw ra, 12(sp)                /* Restore return address after
    drawing. */
151     addi sp, sp, 16              /* Free stack frame. */
152     ret                          /* Return to main loop. */

```

Listing C.1. examples/games/pong/graphics/game.S
complete graphics Pong in RISC-V assembly (Montella et al., 2026f).

C.2 Pong in ASCII assembly

The ASCII variant uses the same three-function structure but renders to the text console, making it the gentlest possible introduction to the calling convention, as used in Chapter 4. Note how closely its `update` mirrors the graphics version: the same input read, the same bit tests, the same clamps, differing only in units (text columns rather than pixels).

```

1  /*
2   * PONG ASCII for PRG32.
3   * This file is intentionally verbose: every instruction is
4   * commented so
5   * first-year students can connect RISC-V assembly to visible
6   * hardware effects.
7   */
8  .option norelax
9  .section .data
10 pong_ascii_x:      .word 20          /* Object x coordinate in
    text columns. */
11 pong_ascii_y:      .word 12          /* Object y coordinate in
    text rows. */
12 pong_ascii_dx:      .word 2          /* Horizontal velocity (
    teaching variable). */
13 pong_ascii_dy:      .word 1          /* Vertical velocity (
    teaching variable). */
14 pong_ascii_score:   .word 0          /* Simple score/counter. */
15
16 .section .rodata
17 pong_ascii_title:   .asciz "PONG ASCII\n"
18 pong_ascii_msg:     .asciz "PRG32 RISC-V ASM DEMO\n"
19
20 .section .text
21 .global pong_ascii_init
22 .global pong_ascii_update
23 .global pong_ascii_draw
24
25 pong_ascii_init:
26     addi sp, sp, -16                /* Build a tiny stack frame
    before calling C. */
27     sw ra, 12(sp)                  /* Save return address because
    call overwrites ra. */
28     call prg32_console_clear        /* Clear UART/LCD text console.
    */
29     la a0, pong_ascii_title         /* Put address of title string
    into argument a0. */
30     call prg32_console_write        /* Print title through

```

```

framework console. */
29     la a0, pong_ascii_msg           /* Load second message pointer
    into a0. */
30     call prg32_console_write        /* Print explanatory message.
    */
31     li a0, 120                      /* Choose a short beep
    frequency/duration argument. */
32     li a1, 30                      /* 30 milliseconds: audible but
    not annoying. */
33     call prg32_audio_beep          /* Call C PWM audio helper. */
34     lw ra, 12(sp)                  /* Restore return address. */
35     addi sp, sp, 16                 /* Destroy stack frame. */
36     ret                             /* Return to the C game loop.
    */
37
38 pong_ascii_update:
39     addi sp, sp, -16                /* Save caller state before
    using C functions. */
40     sw ra, 12(sp)                  /* Preserve return address. */
41     call prg32_input_read          /* Read buttons; result appears
    in a0. */
42     mv t0, a0                      /* Copy button bitmask into
    temporary register t0. */
43
44     la t1, pong_ascii_x            /* Load address of x variable.
    */
45     lw t2, 0(t1)                   /* Load current x value. */
46     andi t3, t0, 1                 /* Test LEFT button bit. */
47     beqz t3, 1f                    /* If not pressed, skip left
    movement. */
48     addi t2, t2, -1                /* Move object left by one text
    column. */
49 1:  andi t3, t0, 2                 /* Test RIGHT button bit. */
50     beqz t3, 2f                    /* If not pressed, skip right
    movement. */
51     addi t2, t2, 1                 /* Move object right by one
    text column. */
52 2:  bltz t2, 3f                    /* Clamp negative x back to
    zero. */
53     li t3, 39                      /* Maximum text column. */
54     ble t2, t3, 4f                 /* Keep x when in range. */
55     mv t2, t3                      /* Clamp to right edge. */
56     j 4f
57 3:  li t2, 0                       /* Clamp to left edge. */
58 4:  sw t2, 0(t1)                   /* Store bounded x. */
59
60     la t1, pong_ascii_y            /* Load address of y variable.
    */
61     lw t2, 0(t1)                   /* Load current y value. */
62     andi t3, t0, 4                 /* Test UP button bit. */
63     beqz t3, 5f                    /* If not pressed, skip up
    movement. */
64     addi t2, t2, -1                /* Move object up by one row.
    */
65 5:  andi t3, t0, 8                 /* Test DOWN button bit. */
66     beqz t3, 6f                    /* If not pressed, skip down

```

```

67      movement. */
        addi t2, t2, 1          /* Move object down by one row.
        */
68  6:   bltz t2, 7f             /* Clamp negative y back to
        zero. */
69      li t3, 24               /* Maximum text row. */
70      ble t2, t3, 8f          /* Keep y when in range. */
71      mv t2, t3               /* Clamp to bottom edge. */
72      j 8f
73  7:   li t2, 0               /* Clamp to top edge. */
74  8:   sw t2, 0(t1)           /* Store bounded y. */
75
76      la t1, pong_ascii_score /* Load score variable address.
        */
77      lw t2, 0(t1)           /* Load current score/counter.
        */
78      addi t2, t2, 1          /* Increment counter once per
        frame. */
79      sw t2, 0(t1)           /* Store updated counter. */
80      lw ra, 12(sp)          /* Restore return address. */
81      addi sp, sp, 16         /* Release stack frame. */
82      ret                    /* Return to C scheduler. */
83
84  pong_ascii_draw:
85      addi sp, sp, -16         /* Save state before C drawing
        calls. */
86      sw ra, 12(sp)          /* Save return address. */
87      call prg32_console_clear /* Text games redraw the
        logical ASCII screen. */
88      la a0, pong_ascii_title /* Put title at top of console.
        */
89      call prg32_console_write /* Write title to current
        console backend. */
90      la t0, pong_ascii_score /* Load score address. */
91      lw a0, 0(t0)           /* Load score value for
        debugging. */
92      call prg32_console_hex32 /* Show score/counter as
        hexadecimal. */
93      li a0, 10              /* ASCII newline. */
94      call prg32_console_putc /* Print newline. */
95      lw ra, 12(sp)          /* Restore return address after
        drawing. */
96      addi sp, sp, 16         /* Free stack frame. */
97      ret                    /* Return to main loop. */

```

Listing C.2. examples/games/pong/ascii/game.S
complete ASCII Pong in RISC-V assembly (Montella et al., 2026f).

C.3 Pong in C

The C version, the basis of Chapter 8, expresses the identical game in a fraction of the lines. Compare it directly with the graphics assembly above: every C statement here corresponds to a block of instructions there, which is the lesson of Chapter 11. This version also keeps a second paddle alive with a fallback AI so the two-player screen stays in motion even without

a second player.

```
1  #include "prg32.h"
2
3  static int paddle_x;
4  static int paddle2_x;
5  static int ball_x;
6  static int ball_y;
7  static int ball_dx;
8  static int ball_dy;
9
10 void pong_c_init(void) {
11     paddle_x = 128;
12     paddle2_x = 128;
13     ball_x = 156;
14     ball_y = 96;
15     ball_dx = 2;
16     ball_dy = 1;
17 }
18
19 void pong_c_update(void) {
20     uint32_t input = prg32_input_read();
21     if (input & PRG32_BTN_LEFT) {
22         paddle_x -= 3;
23     }
24     if (input & PRG32_BTN_RIGHT) {
25         paddle_x += 3;
26     }
27     if (paddle_x < 0) {
28         paddle_x = 0;
29     }
30     if (paddle_x > 256) {
31         paddle_x = 256;
32     }
33
34     uint32_t input2 = prg32_input_read_player(2);
35     if (input2 & PRG32_BTN_LEFT) {
36         paddle2_x -= 3;
37     } else if (input2 & PRG32_BTN_RIGHT) {
38         paddle2_x += 3;
39     } else if (ball_x + 4 < paddle2_x + 32) {
40         paddle2_x -= 1;
41     } else if (ball_x + 4 > paddle2_x + 32) {
42         paddle2_x += 1;
43     }
44     if (paddle2_x < 0) {
45         paddle2_x = 0;
46     }
47     if (paddle2_x > 256) {
48         paddle2_x = 256;
49     }
50
51     ball_x += ball_dx;
52     ball_y += ball_dy;
53     if (ball_x < 0 || ball_x > 312) {
54         ball_dx = -ball_dx;
```

```

55     }
56     if (prg32_sprite_hitbox(ball_x, ball_y, 8, 8, paddle2_x, 4,
64, 8)) {
57         ball_dy = 1;
58     }
59     if (prg32_sprite_hitbox(ball_x, ball_y, 8, 8, paddle_x, 188,
64, 8)) {
60         ball_dy = -1;
61     }
62     if (ball_y < 0 || ball_y > 192) {
63         ball_dy = -ball_dy;
64     }
65 }
66
67 void pong_c_draw(void) {
68     prg32_gfx_clear(PRG32_COLOR_BLACK);
69     prg32_gfx_rect(paddle2_x, 4, 64, 8, PRG32_COLOR_CYAN);
70     prg32_gfx_rect(paddle_x, 188, 64, 8, PRG32_COLOR_WHITE);
71     prg32_gfx_rect(ball_x, ball_y, 8, 8, PRG32_COLOR_YELLOW);
72     prg32_gfx_text8(8, 96, "PONG C P1/P2", PRG32_COLOR_GREEN,
PRG32_COLOR_BLACK);
73 }

```

Listing C.3. examples/games/pong/c/game.c
complete Pong in C (Montella et al., 2026f).

C.4 Breakout in C

Breakout is Pong with one new idea: a row of bricks, stored as the bits of a single byte, each set bit meaning a brick is still present. When the ball reaches the brick row, the code clears the corresponding bit with a bitwise AND and bounces the ball. This is the bitmask technique of Chapter 5 reused for game state, as discussed in Chapter 7.

```

1  #include "prg32.h"
2
3  static int paddle_x;
4  static int ball_x;
5  static int ball_y;
6  static int ball_dx;
7  static int ball_dy;
8  static uint8_t bricks;
9
10 void breakout_c_init(void) {
11     paddle_x = 128;
12     ball_x = 150;
13     ball_y = 120;
14     ball_dx = 2;
15     ball_dy = -2;
16     bricks = 0xff;
17 }
18
19 void breakout_c_update(void) {
20     uint32_t input = prg32_input_read();

```

```

21     if (input & PRG32_BTN_LEFT) {
22         paddle_x -= 3;
23     }
24     if (input & PRG32_BTN_RIGHT) {
25         paddle_x += 3;
26     }
27     if (paddle_x < 0) {
28         paddle_x = 0;
29     }
30     if (paddle_x > 256) {
31         paddle_x = 256;
32     }
33
34     ball_x += ball_dx;
35     ball_y += ball_dy;
36     if (ball_x < 0 || ball_x > 312) {
37         ball_dx = -ball_dx;
38     }
39     if (ball_y < 0 || ball_y > 192) {
40         ball_dy = -ball_dy;
41     }
42     if (prg32_sprite_hitbox(ball_x, ball_y, 8, 8, paddle_x, 184,
64, 8)) {
43         ball_dy = -2;
44     }
45     if (ball_y < 48 && ball_x >= 0 && ball_x < 320 && bricks) {
46         bricks &= (uint8_t)~(1u << ((unsigned)ball_x / 40u));
47         ball_dy = 2;
48     }
49 }
50
51 void breakout_c_draw(void) {
52     prg32_gfx_clear(PRG32_COLOR_BLACK);
53     for (int i = 0; i < 8; ++i) {
54         if (bricks & (1u << i)) {
55             prg32_gfx_rect(i * 40 + 2, 32, 36, 10,
PRG32_COLOR_CYAN);
56         }
57     }
58     prg32_gfx_rect(paddle_x, 184, 64, 8, PRG32_COLOR_WHITE);
59     prg32_gfx_rect(ball_x, ball_y, 8, 8, PRG32_COLOR_YELLOW);
60     prg32_gfx_text8(8, 8, "BREAKOUT C", PRG32_COLOR_GREEN,
PRG32_COLOR_BLACK);
61 }

```

Listing C.4. examples/games/breakout/c/game.c
complete Breakout in C, showing bricks as a bitmask (Montella et al., 2026f).

C.5 Where to find the rest

The PRG32 repository contains, for each of the nine example games, an ASCII assembly version, a graphics assembly version, and a C version, alongside focused feature demos for scrolling, parallax, animated sprites, and dual playfields (Montella et al., 2026f). Having read the four listings above, you have the vocabulary to read any of them. The recommended

next reads, in increasing richness, are Space Invaders, Tetris, the platformer, the raycaster, and Wing Commander, each of which is a variation on the patterns reproduced here.

CHAPTER D

Using GitHub for Your Coursework

The PRG32 platform lives on GitHub, and so should your work ([Montella et al., 2026f](#)). This appendix is a practical introduction to the version-control workflow you will use to obtain the platform, track your own changes, recover from mistakes, and submit assignments. It assumes no prior experience with Git. The aim is not Git mastery but the handful of operations that make coursework safer and submission painless.

D.1 Why version control, concretely

Two everyday disasters motivate everything in this appendix. The first is the embedding lab of Chapter 10, where you temporarily modify the firmware’s build list and main loop, then must restore them; without a record of the original, “restore” becomes guesswork. The second is the moment, familiar to every programmer, when a working game stops working after an edit and you cannot remember what you changed. Version control solves both: it remembers every committed state of your files and lets you compare or return to any of them.

Concept

Git records snapshots, called *commits*, of your project over time. Each commit is a labelled, recoverable state. **GitHub** hosts your Git repository online, so your work is backed up, shareable, and submittable. Together they turn “I think I broke something” into “let me see exactly what I changed.”

D.2 Getting the platform

You obtain PRG32 by *cloning* its repository, which downloads the full project and its history ([Montella et al., 2026f](#)).

```
git clone https://github.com/riscv-prg32/PRG32.git
cd PRG32
```

Listing D.1. Cloning the platform repository.

To incorporate later updates from the upstream project without losing your own work, you fetch and merge changes; for coursework, however, the cleaner path is to keep your own changes on a branch, described below.

D.3 The everyday cycle

Three commands cover the great majority of daily use: `status` to see what has changed, `add` to stage the changes you want to keep, and `commit` to record them with a message.

```
git status                # what has changed?
git add examples/games/mygame/  # stage the files to keep
git commit -m "Add boundary clamp to mygame paddle"
```

Listing D.2. The everyday Git cycle: inspect, stage, commit.

Watch Out

Write commit messages that your future self will understand: “Add boundary clamp to mygame paddle,” not “stuff” or “fix.” A week later, when you are hunting for the commit where a bug appeared, descriptive messages are the difference between a minute and an hour. The platform’s own history follows this practice.

D.4 Branches: experiment freely

A *branch* is an independent line of work. Before a risky change, such as the firmware embedding of Chapter 10, or a large refactor of your game, make a branch. If the experiment fails, you discard the branch and your main line is untouched; if it succeeds, you merge it back.

```
git switch -c experiment-embed-game  # create and switch to a
    branch
# ... make changes, commit them ...
git switch main                      # return to the safe main
    line
git merge experiment-embed-game      # keep the experiment, OR
git branch -D experiment-embed-game  # discard it entirely
```

Listing D.3. Working on a branch, then merging or discarding.

Concept

A branch lets you try something that might break your project, in isolation. The embedding lab is the textbook case: branch first, embed and test, then either merge the working result or discard the branch to restore the firmware to its original state in one command: exactly the “restore afterwards” the tutorial requires ([Montella et al., 2026k](#)).

D.5 Recovering from mistakes

When an edit breaks a working game, Git shows you precisely what changed since your last commit, and can undo it.

```
git diff                                # show every uncommitted
change
git restore examples/games/mygame/game.S # discard changes to
one file
```

Listing D.4. Seeing and undoing uncommitted changes.

The discipline that makes this powerful is committing often: after each working step of the incremental method this book teaches. If you commit after every verified change (“draw the paddle,” “add input,” “add the clamp”), then any break is, at worst, one step’s worth of work to inspect or undo.

D.6 Submitting your work

For submission you push your commits to your own GitHub repository, then share its link or open a pull request as your course directs. You create an empty repository on GitHub, connect it as a remote, and push.

```
git remote add origin https://github.com/<your-user>/<your-repo>.
git
git push -u origin main
```

Listing D.5. Publishing your work to your own GitHub repository.

Historical Note

The PRG32 repository models the conventions you are learning: a **README** that orients a newcomer, a **CONTRIBUTING** guide, a **LICENSE**, a **CITATION.cff** so the work can be cited, and continuous-integration workflows that run automated checks on every change (Montella et al., 2026f). When you structure your own coursework repository, these are the files that signal care and make your work easy for an instructor to evaluate.

D.7 Citing the platform

If you use PRG32 in a report, a thesis, or any submitted work, cite it. The repository ships citation metadata in **CITATION.cff** for exactly this purpose (Montella et al., 2026f). This book cites the platform throughout, and the bibliography entry for the repository is the model to follow in your own writing.

Checkpoint

You should now be able to clone the platform, see what you have changed, stage and commit changes with a meaningful message, work safely on a branch, undo an unwanted change, and push your work to GitHub for submission. These few operations are enough for an entire first year of coursework.

CHAPTER E

Building the Physical PRG32

Everything in this book runs on the QEMU emulator, so a board is never strictly required. But the platform comes alive on real hardware: real buttons, a real display, a real buzzer. Building it is an instructive exercise in its own right. This appendix is the classroom reference wiring for a breadboard PRG32, drawn from the platform’s hardware notes (Montella et al., 2026f,c). Treat it as a parts list and a wiring map, not as a soldering course; the connections are all low-voltage and breadboard-friendly.

E.1 The parts

A working prototype needs a small, inexpensive set of components (Montella et al., 2026f).

Qty	Item	Purpose
1	ESP32-C6 development board	the RISC-V host that runs everything
1	ILI9341 2.8" SPI TFT, 320×240	the video display
1	5-way digital joystick module	player-one input
1	passive buzzer	sound
1	setup button (optional)	force setup mode without holding A+B
1	second joystick (optional)	player-two games such as Pong
1	MAX98357A + 4–8 Ω speaker (optional)	high-quality mono audio
–	breadboard and jumper wires	to connect it all

Table E.1. The breadboard parts list (Montella et al., 2026f).

Concept
The ESP32-C6 is the RISC-V microcontroller at the centre of the board: the same 32-bit RISC-V processor family your code targets on QEMU. Everything else, such as the display, the joystick, the buzzer, is a peripheral wired to its general-purpose pins. Building the board is, concretely, connecting peripherals to a RISC-V processor.

E.2 Display and input wiring

The reference wiring matches the firmware’s pin configuration in `main/prg32_config.h`; using these exact pins means the firmware works without reconfiguration (Montella et al.,

2026c). The display uses the SPI bus; each joystick direction is a switch to ground, and the firmware enables internal pull-ups so no external resistors are needed.

ESP32-C6	ILI9341 display	ESP32-C6	Input
3V3	VCC	GPIO18	P1 LEFT
GND	GND	GPIO19	P1 RIGHT
GPIO7	MOSI	GPIO3	P1 UP
GPIO2	MISO	GPIO13	P1 DOWN
GPIO6	SCLK	GPIO20	P1 SELECT
GPIO10	CS	GPIO21	P1 A
GPIO8	DC	GPIO22	P1 B
GPIO9	RST	GPIO15	passive buzzer
GPIO5	BL		

Table E.2. Reference display and player-one input wiring (Montella et al., 2026c). Each joystick direction connects as a normally-open switch to ground.

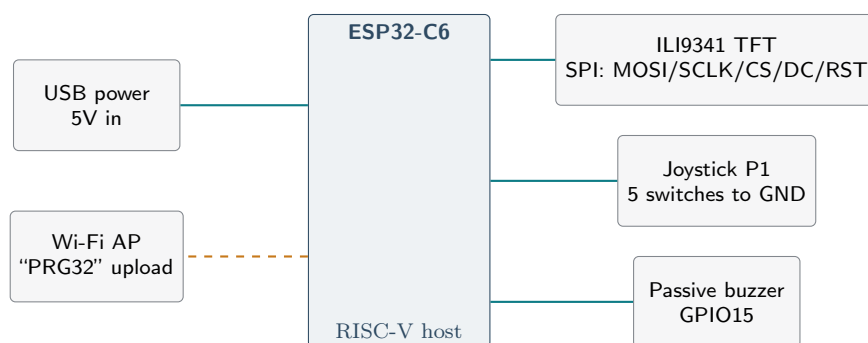


Figure E.1. The reference PRG32 board: a RISC-V ESP32-C6 with a display, a joystick, and a buzzer on its GPIO pins, powered over USB and exposing a Wi-Fi access point for cartridge upload (Montella et al., 2026c,f).

Watch Out

The default audio pins overlap the reference display wiring, and many ESP32-C6 boards wire their onboard RGB LED to GPIO8: the same pin used here for the display’s data/command line. If you add the optional MAX98357A audio or enable the onboard LED, reassign the conflicting pins in `main/prg32_config.h` or `menuconfig` before flashing, or the firmware will skip the feature to protect the display (Montella et al., 2026c).

E.3 Optional high-quality audio

The passive buzzer on GPIO15 is enough for the event beeps used throughout the book. For richer sound, the platform supports a MAX98357A I2S amplifier driving a small speaker, and even a stereo “PRG32 Audio Plus” configuration with two amplifiers on the same bus (Montella et al., 2026c). The platform’s safety note is worth repeating: connect the amplifier only to a 4–8 Ω speaker, never to headphones or a line input, and keep speaker wires short on the breadboard.

E.4 Controllers over USB

The ESP32-C6 does not host arbitrary USB game controllers directly. The platform bridges them: a small helper board (an ESP32-S3, an RP2040 with USB-host support, or a PC-side serial helper) reads the USB controller and forwards button presses to the ESP32-C6 over a simple serial packet ([Montella et al., 2026f](#)). Because that packet feeds the same input bitmask your games already read, no game code changes to gain controller support: a clean example of an abstraction boundary doing its job.

E.5 From breadboard to board

The repository’s **hardware** directory contains starter design files for those who wish to go further than a breadboard: a KiCad project for a printed circuit board, an OpenSCAD enclosure, and a scaffold for a next hardware revision ([Montella et al., 2026f](#)). These are beyond a first course, but they show the path from a classroom prototype to a finished device, and they are open for you to study and extend.

Checkpoint

With the parts above and the wiring of Table E.2, you can assemble a working PRG32 on a breadboard, flash the resident firmware once (Appendix 3), and then upload cartridge after cartridge over the board’s Wi-Fi. Every game you built on QEMU in this book will then run on hardware you wired yourself.

Bibliography

Ernest Cline. *Ready Player One*. Crown Publishers, New York, 2011.

Digital Autonomy with RISC-V in Europe (DARE) Consortium. Digital Autonomy with RISC-V in Europe (DARE). Project announcement, 2025. Six-year initiative funded through the EuroHPC Joint Undertaking.

EuroHPC Joint Undertaking. Advancing European Sovereignty in HPC with RISC-V: the DARE Project. Press release, European High-Performance Computing Joint Undertaking, 2025. <https://eurohpc-ju.europa.eu/>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 ABI: Calling Convention and Runtime Symbols. Documentation, `docs/abi.md`, PRG32 repository, 2026a. <https://github.com/riscv-prg32/PRG32/blob/main/docs/abi.md>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 Cartridge Format. Documentation, `docs/cartridge-format.md`, PRG32 repository, 2026b. <https://github.com/riscv-prg32/PRG32/blob/main/docs/cartridge-format.md>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 Hardware: Reference Wiring. Documentation, `docs/hardware.md`, PRG32 repository, 2026c. <https://github.com/riscv-prg32/PRG32/blob/main/docs/hardware.md>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 Performance Metrics. Documentation, `docs/metrics_api.md`, PRG32 repository, 2026d. https://github.com/riscv-prg32/PRG32/blob/main/docs/metrics_api.md.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 QEMU Screen Emulator. Documentation, `docs/qemu.md`, PRG32 repository, 2026e. <https://github.com/riscv-prg32/PRG32/blob/main/docs/qemu.md>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32: Platform for RISC-V Gaming Development (32-bit). Software repository, University of Naples “Parthenope”, 2026f. <https://github.com/riscv-prg32/PRG32>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. Measuring PRG32 for a Scientific Paper. Documentation, `docs/scientific_measurement_tutorial.md`, PRG32 repository, 2026g. <https://github.com/riscv-prg32/PRG32>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. Teaching with PRG32. Documentation, `docs/teaching_with_prg32.md`, PRG32 repository, 2026h. <https://github.com/riscv-prg32/PRG32>.

Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. PRG32 Tutorial. Documenta-

- tion, docs/tutorial.md, PRG32 repository, 2026i. <https://github.com/riscv-prg32/PRG32/blob/main/docs/tutorial.md>.
- Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. Tutorial: Writing an ASCII Game in RISC-V Assembly. Documentation, docs/tutorial_ascii_game.md, PRG32 repository, 2026j. <https://github.com/riscv-prg32/PRG32>.
- Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. Tutorial: Writing a PRG32 Game in C. Documentation, docs/tutorial_c_game.md, PRG32 repository, 2026k. <https://github.com/riscv-prg32/PRG32>.
- Raffaele Montella, Ivan Cafiero, and Simone Boscaglia. Tutorial: Writing a Graphic Game in RISC-V Assembly. Documentation, docs/tutorial_graphic_game.md, PRG32 repository, 2026l. <https://github.com/riscv-prg32/PRG32>.
- RISC-V International. The RISC-V Instruction Set Manual. Technical specification, RISC-V International, 2024. <https://riscv.org/technical/specifications/>.
- Florian Wohlrab. Introducing the European Unified RISC-V IP Access Platform (UAP). HiPEAC, 2026. <https://www.hipeac.net/news/7137/>.